

Statistical Learning

Artificial Intelligence

Jacopo Parretti

II Semester 2025-2026

Contents

I	Introduction to Statistical Learning	7
1	Motivations	7
1.1	What is Statistical Learning?	7
1.2	A Brief History	7
1.3	Why Study Statistical Learning?	7
1.4	Real-World Applications	8
2	Key Elements of Statistical Learning	8
2.1	The Core Idea	8
2.2	Supervised Learning	9
2.3	Unsupervised Learning	9
2.4	Reinforcement Learning	9
2.5	Basic Concepts and Terminology	9
3	Why Estimate f?	9
3.1	The General Setup	9
3.2	Prediction	9
3.3	Inference	10
3.4	Estimating f : General Setup	10
4	How Do We Estimate f?	10
4.1	Parametric Methods	10
4.2	Non-Parametric Methods	11
5	Prediction Accuracy vs. Model Interpretability	11
6	Assessing Model Accuracy	11
6.1	The Problem of Model Selection	11
6.2	Mean Squared Error	12
6.3	The Bias-Variance Trade-off	12
II	Linear Regression	12
7	Simple Linear Regression	13
7.1	The Model	13
7.2	Estimating the Coefficients via Least Squares	13

7.3	The Population Regression Line	13
7.4	Bias and Standard Error	14
7.5	Confidence Intervals	14
7.6	Hypothesis Testing	14
7.7	Assessing Model Accuracy: RSE and R^2	15
8	Multiple Linear Regression	15
8.1	The Model	15
8.2	Estimating the Coefficients	15
8.3	Important Questions in Multiple Regression	16
9	Considerations about the Regression Model	17
9.1	Qualitative Predictors	17
9.2	Extensions of the Linear Model	17
9.3	Potential Problems	18
III	Cross-Validation	19
10	Resampling Methods	19
11	Test Error vs. Training Error	19
12	The Validation Set Approach	20
12.1	Drawbacks	20
13	Leave-One-Out Cross-Validation (LOOCV)	20
13.1	Advantages over the Validation Set Approach	20
14	k-Fold Cross-Validation	21
14.1	Advantages of k -fold CV over LOOCV	21
14.2	Bias-Variance Trade-off in Cross-Validation	21
14.3	CV Curves and the True Test MSE	22
IV	Linear Model Selection	22
15	Introduction	22
15.1	Why Use Alternative Fitting Procedures?	22
15.2	The Prediction Accuracy vs. Interpretability Trade-off	22
15.3	Three Classes of Approaches	22
16	Best Subset Selection	23
16.1	Algorithm	23
16.2	Computational Limitation	23
17	Forward Stepwise Selection	23
17.1	Algorithm	23
17.2	Computational Cost	24
18	Backward Stepwise Selection	24
18.1	Algorithm	24
18.2	Comparison of the Three Algorithms	24

19 Choosing the Optimal Model	24
19.1 Approach 1: Adjusting the Training Error	24
19.2 Approach 2: Direct Estimation of Test Error	25
V Regularization	25
20 Shrinkage Methods	26
21 Ridge Regression	26
21.1 Formulation	26
21.2 The Tuning Parameter λ	26
21.3 Scale Sensitivity and Standardization	26
21.4 The ℓ_2 Norm Perspective	27
21.5 Bias-Variance Trade-off in Ridge Regression	27
21.6 When to Use Ridge Regression	27
22 The Lasso	27
22.1 Motivation	27
22.2 Formulation	27
22.3 Variable Selection	28
23 Constrained Formulation	28
23.1 Geometric Interpretation	28
24 Lasso vs Ridge Regression	29
25 Selecting the Tuning Parameter	29
VI Dimension Reduction	29
26 Dimension Reduction Methods	29
26.1 The General Framework	29
26.2 Why Dimension Reduction?	30
26.3 Connection to Linear Regression	30
27 Principal Components Analysis	30
27.1 Overview	30
27.2 Principal Component Directions	31
27.3 The First Principal Component	31
27.4 The Second Principal Component	31
28 Principal Components Regression	32
28.1 The PCR Approach	32
28.2 Advantages of PCR	32
28.3 Choosing M via Cross-Validation	32
28.4 PCR is Not Feature Selection	33
28.5 PCR vs Ridge Regression	33
29 Partial Least Squares	33
29.1 Motivation: The Supervised Alternative to PCA	33
29.2 Computing PLS Directions	33
29.3 Fitting and Tuning	34

30	Considerations in High Dimensions	34
30.1	Low-Dimensional vs High-Dimensional Data	34
30.2	What Goes Wrong in High Dimensions?	34
30.3	The Curse of Dimensionality in Practice	35
30.4	Implications for Model Selection Criteria	35
VII	Classification	35
31	Introduction to Classification	36
31.1	Qualitative Variables	36
31.2	The Classification Setting	36
31.3	The Bayes Classifier	36
32	K-Nearest Neighbours	37
32.1	The KNN Classifier	37
32.2	Choice of K : Bias-Variance Trade-off	37
VIII	Classification II — Logistic Regression	38
33	Why Not Linear Regression?	38
33.1	The Problem with Encoding Qualitative Responses	38
33.2	The Binary Case: Linear Regression on a Dummy Variable	38
34	Logistic Regression	39
34.1	The Logistic Model	39
34.2	The Odds	39
34.3	The Logit (Log-Odds)	39
34.4	Estimating the Coefficients	40
35	Multiple Logistic Regression	41
35.1	Extending to Multiple Predictors	41
35.2	Example: Default Dataset with Three Predictors	41
IX	Classification III — Linear Discriminant Analysis	41
36	Introduction	42
36.1	An Alternative Approach to Classification	42
36.2	Why Use LDA?	42
37	Bayes' Theorem for Classification	42
37.1	Setup	42
37.2	Bayes' Theorem	42
38	Linear Discriminant Analysis for $p = 1$	43
38.1	Gaussian Assumption	43
38.2	Discriminant Functions	43
38.3	The LDA Algorithm	44
38.4	Decision Boundary and Performance	44
39	Linear Discriminant Analysis for $p > 1$	44
39.1	Multivariate Gaussian Distribution	44

39.2 Discriminant Functions for $p > 1$	45
39.3 Example: Two Predictors, Three Classes	45
40 Evaluating Classification Performance	45
40.1 Beyond the Overall Error Rate	45
40.2 The Confusion Matrix	46
40.3 Key Metrics	46
40.4 Adjusting the Decision Threshold	46
40.5 The ROC Curve and AUC	47
X Tree-Based Methods	47
41 Introduction to Tree-Based Methods	48
41.1 Overview	48
41.2 Basic Terminology	48
42 Regression Trees	49
42.1 Tree-Building Process	49
42.2 Recursive Binary Splitting	49
42.3 Predictions	49
42.4 Pruning the Decision Tree	50
42.5 Cost-Complexity (Weakest Link) Pruning	50
42.6 Regression Tree Procedure	51
43 Classification Trees	51
43.1 From Regression to Classification	51
43.2 Gini Index and Entropy	51
43.3 Linear Models vs. Tree-Based Approaches	52
43.4 Benefits and Drawbacks of Tree-Based Methods	52
44 Bootstrapping and Bagging	52
44.1 The Bootstrap Concept	52
44.2 Estimating $\text{Var}(\hat{\alpha})$	53
44.3 Bagging	53
44.4 Bagging for Decision Trees	53
44.5 Out-Of-Bag (OOB) Error Estimation	54
44.6 Variable Importance	54
45 Random Forest and Boosting	54
45.1 Random Forest	54
45.2 Boosting	55
45.3 Boosting Algorithm for Regression Trees	55
45.4 Tuning Parameters for Boosting	55
45.5 Boosting for Classification: AdaBoost	55
45.6 Summary	57
XI Unsupervised Learning	57
46 Introduction to Unsupervised Learning	57
46.1 Refresher: Supervised, Unsupervised, Reinforcement Learning	57
46.2 Challenges of Unsupervised Learning	57

47 Principal Component Analysis (Revised)	58
47.1 Overview	58
47.2 First Principal Component	58
47.3 Computing the First Principal Component	58
47.4 Further Principal Components	58
47.5 Example: USA Arrest Dataset	59
47.6 PCA as the Closest Linear Surface	59
47.7 Proportion of Variance Explained (PVE)	59
47.8 How Many Components to Use?	60
47.9 Scaling the Variables	60
48 Clustering Methods: K-means Clustering	60
48.1 What is Clustering?	60
48.2 Two Main Clustering Methods	60
48.3 K-means: Setup	60
48.4 Within-Cluster Variation and Objective	61
48.5 K-means Algorithm	61
48.6 Properties of the K-means Algorithm	61
49 Hierarchical Clustering	62
49.1 Overview	62
49.2 Hierarchical Clustering Algorithm	62
49.3 Types of Linkage	62
49.4 Choice of Dissimilarity Measure	63
49.5 Practical Issues	63
49.6 Example: Breast Cancer Microarray Study	63
49.7 Conclusions	63

Part I

Introduction to Statistical Learning

1 Motivations

1.1 What is Statistical Learning?

Statistical Learning

Statistical learning refers to a vast set of **tools for understanding data**. These tools enable us to model, predict, and draw inferences from complex datasets across a wide range of application domains.

The central goal is to **understand correlations between data**. Depending on what we want to predict and what kind of output we have, this takes three main forms:

- **Regression:** predicting a *continuous* (quantitative) output variable.
Example: predicting wage as a function of age, year of experience, and education level.
- **Classification:** predicting a *discrete* (categorical) output variable.
Example: predicting the direction of S&P 500 daily movement (up/down) from previous days' percentage changes.
- **Clustering:** *grouping* data points based on observed characteristics, with no output label available.
Example: grouping cell lines by gene expression profiles (unsupervised — no cancer type label used during grouping).

1.2 A Brief History

- **Early 1800s** — Legendre and Gauss: least squares method, linear regression.
- **Early 1936** — Fisher: Linear Discriminant Analysis, classification.
- **Mid 1980s** — Breiman, Friedman, Olshen, Stone: classification and regression trees; tree-based methods and cross-validation for model selection.
- **Present** — Machine Learning, Data Science, Deep Learning.

1.3 Why Study Statistical Learning?

Statistical learning methods are key assets for a wide range of disciplines:

- Artificial Intelligence, Machine Learning, Economics, Business, Social Sciences, Medicine, Biotechnology, Public Policies
- Finance, Manufacturing, Pharmaceutical, Logistics, Entertainment, Politics, Tourism

Three core principles motivate this course:

1. Statistical learning methods should **not** be treated as black boxes: to select the best method you must understand how they work.
2. You do not need to re-implement state-of-the-art methods, but you must **understand them and know when and how to use them**.

3. The goal is to develop competence in the **application of statistical learning methods to real-world problems** (data science).

1.4 Real-World Applications

Two representative application areas illustrate the scope of the discipline.

1.4.1 District Heating Networks

Goal: predict heating load in the next 48 hours.

- **Data:** multivariate time series (heat demand of previous days, external temperature, humidity, ...)
- **Model:** a set of linear autoregressive models
- **Output:** load demand forecast for the next 48 hours

This is a regression problem. Evaluation metrics used in practice include RMSE and MAPE across different regimes (weekdays, weekends, holidays).

1.4.2 Environmental Monitoring — Autonomous Water Drones

Goal: recognize what a drone is doing from its sensor stream (activity recognition).

- **Data:** multivariate time series from GPS, engine control, battery, electrical conductivity, dissolved oxygen, temperature sensors
- **Task:** find time intervals representing important situations (e.g., drone moving against current)
- **Approach:** unsupervised learning — K-means, Affinity Propagation, Gaussian Mixture Models, Hidden Markov Models

Key result: it is possible to **segment time series** and identify meaningful clusters without knowing the labels. For upstream/downstream navigation, K-means identifies average direction of movement, average voltage to motors, and standard deviation of battery voltage as the most relevant features.

2 Key Elements of Statistical Learning

2.1 The Core Idea

Core idea

Statistical learning is about **automatically building models starting from data**. In all cases, what we learn is a **function**.

There are three main types of learning, distinguished by what information is available during training:

- **Supervised learning:** $y = f(x)$ — given pairs (x, y) , learn $f(\cdot)$ [function approximation].
- **Unsupervised learning:** $f(x)$ — given only x , learn $f(\cdot)$ as a compact representation of x [clustering].
- **Reinforcement learning:** $y = f(x), z$ — given (x, z) , learn $f(\cdot)$ that generates y , where x is the state and z is the reward signal [sequential decision making].

2.2 Supervised Learning

In supervised learning we have:

- A **training set** of labeled examples: known inputs x and corresponding labels t .
- A **model** $y(x)$ that maps inputs to predicted outputs.
- A **test set** of new, unseen inputs: the model is applied to evaluate generalization.

Example: recognizing hand-written digits. Training data consists of images x and digit labels t . The model learns a mapping $y(x)$; at test time it predicts the digit for new images.

2.3 Unsupervised Learning

In unsupervised learning there are **no labels**. The goal is to build a model that **groups similar** data points — by columns, rows, or both (bi-clustering).

Example: gene expression data. Rows are genes, columns are cell lines, each entry is the expression level. No cancer-type label is used; the task is to discover structure in the data.

2.4 Reinforcement Learning

Reinforcement learning is a **control problem with sequential decision making**:

- A **reward signal** drives the agent's actions (e.g., high reward when reaching a target).
- The agent must **interact with the environment** to learn.
- It uses data-driven statistical techniques but the focus is on **control**, not data analysis.

2.5 Basic Concepts and Terminology

- **Input variables** (independent variables, predictors, features): denoted X when scalar, $X_1, X_2, \dots, X_p$ when there are p predictors.
- **Output variable** (response, dependent variable): denoted Y .
- **Quantitative variables**: numerical values (temperature, height, income, ...).
- **Qualitative variables** (categorical): values from one of K classes (gender, brand, ...).
- **Regression**: quantitative output.
- **Classification**: qualitative output.

3 Why Estimate f ?

3.1 The General Setup

Assume $Y = f(X) + \epsilon$, where f is an unknown function of $X = (X_1, X_2, \dots, X_p)$ and ϵ is a **random error term** independent of X . Statistical learning is concerned with estimating f .

There are two main motivations for estimating f : **prediction** and **inference**.

3.2 Prediction

When inputs X are easy to observe but output Y is difficult to measure directly, we build a model $\hat{f}(\cdot)$ to compute:

$$\hat{Y} = \hat{f}(X)$$

In prediction, the exact form of $\hat{f}$ is not of primary interest — it acts as a **black box** as long as predictions are accurate.

3.2.1 Reducible and Irreducible Error

The accuracy of $\hat{Y}$ as a prediction of Y depends on two quantities:

- **Reducible error:** error that arises because $\hat{f}$ is not a perfect estimate of f . This can in principle be reduced by using better models or more data.
- **Irreducible error:** variability of ϵ (the random error), which is independent of X and cannot be eliminated regardless of how well we estimate f .

Formally, the expected squared prediction error decomposes as:

$$E[(Y - \hat{Y})^2] = E[(f(X) - \hat{f}(X))^2] + \text{Var}(\epsilon) = \underbrace{E[(f(X) - \hat{f}(X))^2]}_{\text{Reducible error}} + \underbrace{\text{Var}(\epsilon)}_{\text{Irreducible error}}$$

where $E(X)$ is the expected value (population average) of X and $\text{Var}(X)$ is its variance.

3.3 Inference

When the goal is to **understand how Y changes as X changes**, prediction accuracy is secondary — we need to know the **form** of $f(\cdot)$, so $\hat{f}$ cannot be a black box.

Typical questions in inference:

- Which independent variables are **associated** with the response? (key for interpretability)
- What is the **relationship** between the response and each predictor: positive or negative effect?
- Can the relationship between Y and X be captured by a **linear model**? Linear models are simpler and easier to use but have limited representational power.

3.4 Estimating f : General Setup

Given:

- n training data-points, p predictors
- x_{ij} : value of observation i for predictor j , where $i = 1, \dots, n$ and $j = 1, \dots, p$
- y_i : response value for observation i
- Training set: $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ where $x_i = (x_{i1}, x_{i2}, \dots, x_{ip})^T$

Goal: find $\hat{f}(\cdot)$ such that $Y \approx \hat{f}(X)$ for any observation (X, Y) .

4 How Do We Estimate f ?

4.1 Parametric Methods

Parametric methods follow a **two-step procedure**:

1. **Assume a form** for $f(\cdot)$. For example, assume f is *linear*:

$$f(X) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p$$

A model is then completely identified by $p + 1$ parameters $\beta_0, \beta_1, \dots, \beta_p$.

2. **Fit or train** the model using the training data. For a linear model this means estimating $\beta_0, \beta_1, \dots, \beta_p$ such that

$$Y \approx \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p$$

The most common approach is **Least Squares**.

The key point: by assuming a parametric form we reduce the problem of estimating an arbitrary function f to estimating a finite set of parameters.

4.2 Non-Parametric Methods

Non-parametric methods make **no assumption on the form of $f(\cdot)$** and instead try to fit f as closely as possible to the training data.

- **Pros:** can approximate the training data very well, no structural assumption.
- **Cons:** require a very large number of observations; can result in **overfitting**.

A typical example is thin-plate splines. A smooth spline fits the data well while generalizing; a rough spline interpolates the training points almost exactly but overfits.

Overfitting

Overfitting occurs when a model fits the training data too closely, capturing noise rather than the true underlying pattern. The model performs well on training data but poorly on new, unseen data.

5 Prediction Accuracy vs. Model Interpretability

There is a fundamental **trade-off between flexibility and interpretability** in statistical learning methods. Along the flexibility axis, from most interpretable to most flexible:

1. **Subset selection, Lasso** — high interpretability, low flexibility
2. **Least Squares** — moderate interpretability
3. **Generalized Additive Models, Trees** — moderate
4. **Bagging, Boosting** — low interpretability, high flexibility
5. **Support Vector Machines** — low interpretability, high flexibility

More flexible methods tend to achieve better **prediction accuracy** on complex data but at the cost of **interpretability**. When inference is the goal, simpler models are preferred; when prediction accuracy is paramount and the model is used as a black box, more flexible methods may be appropriate.

6 Assessing Model Accuracy

6.1 The Problem of Model Selection

No method outperforms all others on all datasets. Selecting the best approach for a given problem is a challenging and crucial task. The key question is: how do we measure **quality of fit** — how far is the predicted response from the true response?

6.2 Mean Squared Error

For regression problems, the standard measure is the **Mean Squared Error (MSE)**:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2$$

- **Training MSE**: MSE computed on the training data. Minimizing training MSE is how models are typically fit.
- **Test MSE**: MSE computed on new, unseen data. This is what we actually care about.

There is **no guarantee** that the method with minimum training MSE will achieve minimum test MSE. A model that fits the training data too well (low training MSE) often performs worse on test data (high test MSE) — overfitting.

6.3 The Bias-Variance Trade-off

The expected test MSE at a given point x_0 can always be decomposed into three components:

$$E[(y_0 - \hat{f}(x_0))^2] = \text{Var}(\hat{f}(x_0)) + [\text{Bias}(\hat{f}(x_0))]^2 + \text{Var}(\epsilon)$$

- **Variance** $\text{Var}(\hat{f}(x_0))$: how much $\hat{f}$ would change if estimated on a different training set. More flexible models have **higher variance**.
- **Bias** $[\text{Bias}(\hat{f}(x_0))]^2$: error introduced by the assumptions made on the model. Simpler models (e.g., linear) have **higher bias**.
- **Irreducible error** $\text{Var}(\epsilon)$: irreducible noise, independent of the model.

Bias-Variance Trade-off

As model flexibility increases, **variance increases** and **bias decreases**. The optimal model minimizes the sum of variance and squared bias. This trade-off is unavoidable — there is no universally best model.

Different datasets exhibit different bias-variance trade-offs (different rates of change). In practice, we cannot compute the test MSE directly, but we can **estimate** it — for example, using **cross-validation**.

Part II

Linear Regression

Linear regression is a useful tool for predicting a **quantitative** response. It is a simple yet powerful approach that introduces the supervised learning perspective, and serves as a jumping-off point for more modern statistical learning methods — many of which can be seen as generalizations or extensions of linear regression.

7 Simple Linear Regression

7.1 The Model

Simple linear regression predicts a quantitative response Y on the basis of a **single predictor** X :

$$Y \approx \beta_0 + \beta_1 X$$

We read $\approx$ as “is approximately modelled as”. The **intercept** β_0 and the **slope** β_1 are the model **coefficients** (or parameters). Once we have estimated $\hat{\beta}_0$ and $\hat{\beta}_1$ from training data, predictions are computed as:

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

Running example: the Advertising dataset contains sales of a product in 200 different markets, together with advertising budgets for TV, radio, and newspaper. We can regress sales Y on TV spending X :

$$\text{sales} \approx \beta_0 + \beta_1 \times \text{TV}$$

7.2 Estimating the Coefficients via Least Squares

β_0 and β_1 are unknown. Starting from n observation pairs $(x_1, y_1), \dots, (x_n, y_n)$, we define the **residual** for observation i as:

$$e_i = y_i - \hat{y}_i$$

the difference between the i -th observed and predicted response. The **Residual Sum of Squares (RSS)** is:

$$\text{RSS} = e_1^2 + \dots + e_n^2 = \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2$$

The least squares estimates minimize the RSS. The closed-form solution is:

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

where $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$ and $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$ are the sample means.

For the TV advertising example: $\hat{\beta}_0 = 7.03$, $\hat{\beta}_1 = 0.0475$.

7.3 The Population Regression Line

Population Regression Line

When f can be approximated by a linear function, the **population regression line** is the model

$$Y = \beta_0 + \beta_1 X + \epsilon$$

that defines the best linear approximation to the **true relationship** between X and Y .

- β_0 : the intercept — expected value of Y when $X = 0$.
- β_1 : the slope — average increase in Y associated with a one-unit increase in X .

The least squares line is an **estimate** of the population regression line, just as the sample mean $\hat{\mu}$ estimates the population mean $\bar{\mu}$. The least squares line differs from the true line but is close, in the same way that the sample mean is different from but close to the population mean.

7.4 Bias and Standard Error

An estimator is **unbiased** if it does not systematically over- or under-estimate the true parameter. The sample mean $\hat{\mu}$ is an unbiased estimator of $\bar{\mu}$: over many datasets, the average of $\hat{\mu}$ coincides with $\bar{\mu}$.

To quantify how far a single estimator $\hat{\mu}$ will be from $\bar{\mu}$, we use the **Standard Error**:

Standard Error

$$\text{Var}(\hat{\mu}) = \text{SE}(\hat{\mu})^2 = \frac{\sigma^2}{n}$$

where σ is the standard deviation of each y_i in Y . The more observations we have, the smaller the standard error.

Analogously, the standard errors of $\hat{\beta}_0$ and $\hat{\beta}_1$ are:

$$\text{SE}(\hat{\beta}_0)^2 = \sigma^2 \left[\frac{1}{n} + \frac{\bar{x}^2}{\sum_{i=1}^n (x_i - \bar{x})^2} \right], \quad \text{SE}(\hat{\beta}_1)^2 = \frac{\sigma^2}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

where $\sigma^2 = \text{Var}(\epsilon)$. Since σ^2 is not known, it is estimated from data via the **Residual Standard Error**:

$$\text{RSE} = \sqrt{\frac{\text{RSS}}{n-2}}$$

7.5 Confidence Intervals

A **95% confidence interval** is a range of values such that there is a 95% probability that the range contains the true unknown value of the parameter. For β_i , assuming a Gaussian distribution, the 95% CI approximately takes the form:

$$\hat{\beta}_i \pm 2 \cdot \text{SE}(\hat{\beta}_i)$$

That is, there is approximately a 95% chance that the interval $[\hat{\beta}_i - 2 \text{SE}(\hat{\beta}_i), \hat{\beta}_i + 2 \text{SE}(\hat{\beta}_i)]$ contains the true value of β_i .

7.6 Hypothesis Testing

Standard errors can also be used to perform **hypothesis tests** on the coefficients. The most common test is:

$$H_0 : \beta_1 = 0 \quad (\text{no relationship between } X \text{ and } Y)$$

versus

$$H_a : \beta_1 \neq 0 \quad (\text{some relationship exists})$$

To test H_0 , we need to assess whether $\hat{\beta}_1$ is sufficiently far from zero. We compute the ***t*-statistic**:

$$t = \frac{\hat{\beta}_1 - 0}{\text{SE}(\hat{\beta}_1)}$$

which measures the number of standard deviations that $\hat{\beta}_1$ is away from 0. We then compute the ***p*-value**: the probability of observing any value of the *t*-statistic equal to $|t|$ or larger, assuming $\beta_1 = 0$. A **small *p*-value** implies there is an association between the predictor and the response, so we can **reject the null hypothesis**. Typical cut-offs are 0.01 and 0.05.

7.7 Assessing Model Accuracy: RSE and R^2

Two standard measures quantify how well the model fits the data.

Residual Standard Error (RSE)

$$\text{RSE} = \sqrt{\frac{1}{n-2} \text{RSS}} = \sqrt{\frac{1}{n-2} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

RSE estimates the standard deviation of ϵ . It measures the **lack of fit** of the linear model to the data, in the units of Y .

R^2 Statistic

$$R^2 = \frac{\text{TSS} - \text{RSS}}{\text{TSS}} = 1 - \frac{\text{RSS}}{\text{TSS}}$$

where $\text{TSS} = \sum (y_i - \bar{y})^2$ is the **total sum of squares**. R^2 is a proportion (between 0 and 1), independent of the scale of Y .

Interpretation:

- TSS measures total variance in Y *before* regression; RSS measures the variance remaining *after* regression.
- R^2 measures the **proportion of variability in Y explained by X** .
- $R^2 \approx 1$: the regression explains most of the variability — good linear fit.
- $R^2 \approx 0$: the regression explains little — the linear model is wrong or σ^2 is high.

In simple linear regression, $R^2 = r^2$ where $r = \text{Cor}(X, Y)$ is the sample correlation:

$$\text{Cor}(X, Y) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

8 Multiple Linear Regression

8.1 The Model

With p predictors, the model generalizes to:

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p + \epsilon$$

β_j represents the **average effect on Y of a one-unit increase in X_j , holding all other predictors fixed**. For the advertising example:

$$\text{sales} = \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{radio} + \beta_3 \times \text{newspaper} + \epsilon$$

8.2 Estimating the Coefficients

$\beta_0, \beta_1, \dots, \beta_p$ are unknown and must be estimated. Given estimates $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p$, predictions are made via:

$$\hat{y} = \hat{f}(X_p) = \hat{\beta}_0 + \sum_{j=1}^p \hat{\beta}_j x_j$$

We choose $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p$ to minimize the RSS:

$$\text{RSS} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n \left(y_i - \hat{\beta}_0 - \sum_{j=1}^p \hat{\beta}_j x_{ij} \right)^2$$

Unlike simple linear regression, the minimizers do not have a simple closed form and are computed via matrix algebra (normal equations).

Remark. Correlation between predictors can make individual coefficient estimates difficult to interpret. In the advertising example, newspaper has a significant positive coefficient in simple regression ($p < 0.001$) but a near-zero, non-significant coefficient ($p = 0.86$) in multiple regression. This is because newspaper is correlated with radio ($r = 0.354$): in markets with high radio budgets, newspaper budgets also tend to be high, and sales are driven by radio, not newspaper.

Correlation $\neq$ causality. A predictor may appear significant in simple regression only because it is correlated with the true driving variable. Multiple regression helps disentangle these effects but does not establish causation.

8.3 Important Questions in Multiple Regression

8.3.1 Question 1: Is at least one predictor useful?

We test the joint null hypothesis $H_0 : \beta_1 = \beta_2 = \dots = \beta_p = 0$ against H_a : at least one $\beta_j \neq 0$ using the *F*-statistic:

$$F = \frac{(\text{TSS} - \text{RSS})/p}{\text{RSS}/(n - p - 1)}$$

where $\text{TSS} = \sum (y_i - \bar{y})^2$ and $\text{RSS} = \sum (y_i - \hat{y}_i)^2$. Small values of F indicate no response-predictor relationship.

8.3.2 Question 2: Which predictors are important?

When p is large, testing all 2^p subsets is infeasible. Three classical automated approaches exist:

- **Forward selection:** start with the null model (intercept only). Fit p simple regressions and add the variable with the lowest RSS. Continue adding variables that most reduce RSS, until a stopping rule is satisfied.
- **Backward selection:** start with all p variables. Remove the variable with the largest p -value (least statistically significant). Refit with $p - 1$ variables and repeat, until all remaining variables have p -value below a threshold.
- **Mixed selection:** start with no variables and add the best fitting variable one at a time. If the p -value for any current variable rises above a threshold as new variables are added, remove it. Continue until all variables in the model have a sufficiently low p -value.

8.3.3 Question 3: Model fit

In multiple regression, the RSE is defined as:

$$\text{RSE} = \sqrt{\frac{1}{n - p - 1} \text{RSS}}$$

Adding more predictors to the model will always increase R^2 (even if the additional predictor is weakly related to the response). Graphical summaries — such as residual plots — can reveal problems not visible from numerical statistics alone. For example, a non-linear pattern in the residuals suggests a **synergy** or **interaction effect** between predictors.

8.3.4 Question 4: Predictions

Three sources of uncertainty are associated with regression predictions:

1. The least squares plane is only an estimate for the true population regression plane (different datasets yield different coefficients).
2. Model bias: assuming a linear $f(X)$ is almost always an approximation of reality (reducible error).
3. Even knowing the true $f(X)$, an irreducible error ϵ is always present — we use **prediction intervals** to account for this.

9 Considerations about the Regression Model

9.1 Qualitative Predictors

9.1.1 Two-Level Qualitative Predictors

When a qualitative predictor (factor) has two levels, we create an **indicator** or **dummy variable** that takes two numerical values. For example, to encode gender in a credit card balance study:

$$x_i = \begin{cases} 1 & \text{if the } i\text{-th person is female} \\ 0 & \text{if the } i\text{-th person is male} \end{cases}$$

The regression equation becomes:

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i = \begin{cases} \beta_0 + \beta_1 + \epsilon_i & \text{if female} \\ \beta_0 + \epsilon_i & \text{if male} \end{cases}$$

Interpretation: β_0 is the average balance among males; $\beta_0 + \beta_1$ is the average among females; β_1 is the average difference in balance between females and males.

9.1.2 Qualitative Predictors with More than Two Levels

A single dummy variable cannot encode more than two levels. For a qualitative predictor with K levels we create $K - 1$ dummy variables. For instance, for a three-level ethnicity variable (Asian, Caucasian, African American):

$$x_{i1} = \begin{cases} 1 & \text{Asian} \\ 0 & \text{otherwise} \end{cases} \quad x_{i2} = \begin{cases} 1 & \text{Caucasian} \\ 0 & \text{otherwise} \end{cases}$$

The model becomes $y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \epsilon_i$, which gives:

$$y_i = \begin{cases} \beta_0 + \beta_1 + \epsilon_i & \text{Asian} \\ \beta_0 + \beta_2 + \epsilon_i & \text{Caucasian} \\ \beta_0 + \epsilon_i & \text{African American} \end{cases}$$

The level with no dummy variable is the **baseline**.

9.2 Extensions of the Linear Model

The standard linear model assumes the relationship between predictors and response is **linear** and **additive**:

- **Additivity**: the effect of X_j on Y is independent of the other predictors.
- **Linearity**: the change in Y due to a one-unit change in X_j is constant.

9.2.1 Interaction Effects

The **synergy** (or **interaction**) effect occurs when the effect of one predictor depends on the value of another. To accommodate it, we include an **interaction term** X_1X_2 :

$$Y = \beta_0 + \beta_1X_1 + \beta_2X_2 + \beta_3X_1X_2 + \epsilon$$

Defining $\tilde{\beta}_1 = (\beta_1 + \beta_3X_2)$, this can be rewritten as $Y = \beta_0 + \tilde{\beta}_1X_1 + \beta_2X_2 + \epsilon$. The effect of X_1 on Y is no longer constant: adjusting X_2 changes the impact of X_1 on Y .

9.2.2 Non-linear Relationships

Polynomial regression extends the linear model to accommodate non-linear relationships while remaining a linear model in the coefficients:

$$\text{mpg} = \beta_0 + \beta_1 \times \text{horsepower} + \beta_2 \times \text{horsepower}^2 + \epsilon$$

The second predictor is $X_2 = \text{horsepower}^2$; the model is still linear in $\beta_0, \beta_1, \beta_2$.

9.3 Potential Problems

9.3.1 1. Non-linearity of the Data

The linear model assumes a linear relationship between X and Y . **Residual plots** (plotting $e_i = y_i - \hat{y}_i$ versus fitted values $\hat{y}_i$) help identify non-linearity: a pattern in the residuals signals that the linear assumption is violated. A simple remedy is to apply **non-linear transformations** of the predictors, such as $\log X$ or $\sqrt{X}$.

9.3.2 2. Correlation of Error Terms

An important assumption is that the error terms $\epsilon_1, \epsilon_2, \dots, \epsilon_n$ are **uncorrelated**. If there is correlation among errors:

- Estimated standard errors will underestimate the true standard error.
- Confidence intervals will be narrower than they should be.
- p -values will be lower than they should be — parameters may be declared erroneously significant.

Correlated errors create an **unwarranted sense of confidence** in the model. This problem frequently arises in time series data.

9.3.3 3. Non-constant Variance of Error Terms

Linear regression also assumes **homoscedasticity**: $\text{Var}(\epsilon_i) = \sigma^2$ (constant). When the variance of the errors increases with the fitted values, this is called **heteroscedasticity**. It is identifiable from a funnel-shaped residual plot. A common solution is to transform the response with a concave function such as $\log(Y)$.

9.3.4 4. Outliers

An **outlier** is a point y_i that is far from the value predicted by the model (arising, for instance, from incorrect recording). Outliers have little effect on the least squares fit itself but can cause problems with **confidence intervals and p -values**: for example, an outlier may inflate the RSE, making all confidence intervals wider and p -values less significant.

9.3.5 5. Collinearity

Collinearity

Collinearity refers to the situation in which two or more predictor variables are closely related to each other.

Collinearity makes it difficult to separate out the individual effects of collinear variables on the response. Geometrically, the RSS contour becomes elongated: there are many pairs $(\hat{\beta}_{\text{limit}}, \hat{\beta}_{\text{rating}})$ with a similar RSS, so the minimum is not well-determined. Consequences:

- Estimates $\hat{\beta}_j$ become very uncertain (high standard errors).
- The t -statistic for each collinear predictor declines, inflating p -values — a truly associated variable may fail to have a small p -value.

Collinearity can be detected from the **correlation matrix** of predictors. When more than two variables are involved, the **Variance Inflation Factor (VIF)** is used; its presence motivates variable selection methods such as those covered in subsequent chapters.

Part III

Cross-Validation

10 Resampling Methods

Resampling methods involve repeatedly drawing samples from a training set and refitting a model of interest on each sample in order to obtain additional information about that model. For example, we can estimate the variability of a linear regression fit by drawing different samples from the training data and fitting a linear regression to each one. The main drawback is computational cost — fitting the same statistical method multiple times using different subsets of the training data — but this is generally not prohibitive.

The two most commonly used resampling methods are **cross-validation** and **bootstrapping**. Cross-validation is used to estimate the test error associated with a given statistical learning method and to select the appropriate level of model flexibility.

11 Test Error vs. Training Error

- The **test error** is the average error that results from using a statistical learning method to predict the response on a **new observation** — a measurement not used for training. A method can be considered warranted if it produces models with low test error.
- The **training error** can be easily calculated by applying the method to the observations used in its training. Unfortunately, the training error rate often **underestimates** the test error rate considerably.

When a dedicated test set is not available, we consider methods that estimate the test error by **holding out** a subset of training observations from the fitting process and then evaluating the error on that held-out subset.

12 The Validation Set Approach

Validation Set Approach

Randomly divide the available observations into two parts: a **training set** and a **validation set** (or hold-out set) of approximately equal size. Fit the model on the training set, then evaluate predictions on the validation set. The resulting **validation set error rate** — typically the MSE — provides an estimate of the test error rate.

12.1 Drawbacks

1. The validation estimate of the test error rate can be **highly variable**, depending on which observations happen to be included in the training set and which in the validation set.
2. Only a subset of observations is used to fit the model. Since statistical methods generally perform better with more data, the **validation set error rate may overestimate** the test error rate for the model fit on the entire dataset.

Example: on the Auto dataset, using the validation set approach to estimate the test MSE for a polynomial regression of mpg on horsepower, the estimated MSE varies substantially across different random splits of the data (right panel of the plot shows the high variability across ten random splits).

13 Leave-One-Out Cross-Validation (LOOCV)

Leave-One-Out Cross-Validation

LOOCV involves splitting the observations into two parts. Instead of two comparable-size subsets, a **single observation** (x_1, y_1) is used as the validation set, and the remaining $n - 1$ observations $(x_2, y_2), \dots, (x_n, y_n)$ form the training set.

The model is fit on $n - 1$ observations and a prediction $\hat{y}_1$ is made for the excluded observation using x_1 . Since (x_1, y_1) was not used in fitting:

$$\text{MSE}_1 = (y_1 - \hat{y}_1)^2$$

is an approximately unbiased estimate of the test error. However, it is a **poor estimate** on its own because it is based on a single observation and hence highly variable.

Repeating the procedure by selecting each (x_i, y_i) in turn as the validation observation produces n squared errors $\text{MSE}_1, \dots, \text{MSE}_n$. The LOOCV estimate for the test MSE averages these:

$$\text{CV}(n) = \frac{1}{n} \sum_{i=1}^n \text{MSE}_i$$

13.1 Advantages over the Validation Set Approach

1. **Far less bias:** each training set contains $n - 1$ observations, almost as many as the full dataset. LOOCV therefore tends not to overestimate the test error rate as much as the validation set approach.
2. **No randomness:** LOOCV always yields the same result — there is no randomness in the training/validation split.

LOOCV is a very general method and can be used with any kind of predictive modeling (logistic regression, linear discriminant analysis, etc.).

14 k-Fold Cross-Validation

k-Fold Cross-Validation

Randomly divide the set of observations into k groups (folds) of approximately equal size.

1. Treat the first fold as the validation set and fit the method on the remaining $k - 1$ folds; compute MSE_1 on the validation fold.
2. Repeat k times, each time using a different fold as the validation set.
3. This yields k estimates $\text{MSE}_1, \dots, \text{MSE}_k$. The k -fold CV estimate is:

$$\text{CV}(k) = \frac{1}{k} \sum_{i=1}^k \text{MSE}_i$$

LOOCV is a special case of k -fold CV in which $k = n$.

14.1 Advantages of k -fold CV over LOOCV

In practice, k -fold CV is typically performed with $k = 5$ or $k = 10$. The advantages over LOOCV are:

1. **Computational:** LOOCV requires fitting the model n times, which can be prohibitively expensive for large n . k -fold CV with $k = 10$ requires fitting only 10 times.
2. **Bias-variance trade-off:** each random split of observations estimates the test error differently. The variability in k -fold CV estimates is lower than the variability in the validation set approach, though higher than LOOCV's variability.

14.2 Bias-Variance Trade-off in Cross-Validation

The choice of k in k -fold CV involves a bias-variance trade-off:

- **Validation set approach:** each training set contains only about half the observations $\Rightarrow$ **high bias** (overestimates test error), low variance issues come from using a single split.
- **LOOCV:** each training set contains $n - 1$ observations $\Rightarrow$ **approximately unbiased** estimates of the test error. However, LOOCV has **higher variance** than k -fold CV.
- **k -fold CV ($k < n$):** each training set contains $(k - 1)n/k$ observations $\Rightarrow$ **intermediate level of bias**.

14.2.1 Why does LOOCV have higher variance?

When performing LOOCV we average the outputs of n fitted models, each trained on an almost identical set of observations. These outputs are therefore **highly (positively) correlated** with each other. In contrast, with k -fold CV ($k < n$) we average the outputs of k fitted models whose training sets overlap less — they are **less correlated**. Since the mean of many highly correlated quantities has higher variance than the mean of less correlated quantities, the test error estimate from LOOCV tends to have higher variance than that from k -fold CV.

From the perspective of **bias reduction**, LOOCV should be preferred to k -fold CV. However, accounting for **variance**, k -fold CV with $k = 5$ or $k = 10$ is typically the preferred choice: these values have been shown empirically to yield test error rate estimates that suffer neither from excessively high bias nor from very high variance.

14.3 CV Curves and the True Test MSE

When examining real data, the true test MSE is unknown, so it is difficult to assess the accuracy of the CV estimate directly. On simulated data, where the true test MSE can be computed, CV curves have the **correct general shape** — they identify the right level of flexibility — but they tend to **underestimate** the true test MSE slightly. For model selection purposes (choosing among competing models or selecting the degree of flexibility), what matters is the shape of the curve rather than its absolute value.

Part IV

Linear Model Selection

15 Introduction

15.1 Why Use Alternative Fitting Procedures?

The standard linear model $Y = \beta_0 + \sum_{j=1}^p \beta_j X_j + \epsilon$ fit by least squares is a workhorse of statistical learning: it offers strong inference guarantees and, on real-world problems, is often surprisingly competitive with non-linear methods. However, plain least squares can be improved by replacing it with **alternative fitting procedures**, for two reasons.

15.1.1 Prediction Accuracy

- If $n \gg p$ (observations much larger than variables), least squares estimates have low variance and perform well on test observations.
- If n is **not much larger than p** , there can be substantial variability in the least squares fit, leading to overfitting and poor predictions on unseen data.
- If $p > n$, there is **no unique least squares estimate**: variance is infinite and the method cannot be used at all.

15.1.2 Model Interpretability

In multiple regression, some predictors are often **not associated with the response**, introducing unnecessary complexity. Setting the corresponding coefficient estimates to zero produces a more interpretable model. This is the goal of **feature selection** (or **variable selection**).

15.2 The Prediction Accuracy vs. Interpretability Trade-off

	Fewer predictors	More predictors
Interpretability	Easy	Difficult
Overfitting risk	Low	High
Prediction accuracy	Low	High
Computational cost	Low	High

15.3 Three Classes of Approaches

There are three main strategies for fitting $Y = \beta_0 + \sum_{i=1}^p \beta_i X_i + \epsilon$ beyond plain least squares:

- **Subset Selection**: select a subset of the p predictors and fit the model by least squares on the reduced set.

- **Shrinkage** (regularization): fit the model on all p predictors but shrink the estimated coefficients towards zero. This reduces variance; some coefficients may be shrunk to exactly zero, performing variable selection implicitly.
- **Dimension Reduction**: project the p predictors into an M -dimensional subspace ($M < p$) by computing M linear combinations of the variables, then fit a linear regression model on these M projections using least squares.

16 Best Subset Selection

16.1 Algorithm

Best subset selection fits a separate least squares regression for **every possible combination** of the p predictors.

1. Let $\mathcal{M}_0$ denote the **null model** — no predictors, predicts the sample mean for every observation.
2. For $k = 1, \dots, p$:
 - (a) Fit all $\binom{p}{k} = \frac{p!}{k!(p-k)!}$ models that contain exactly k predictors.
 - (b) Choose the best among these models (smallest RSS / highest R^2) and call it $\mathcal{M}_k$.
3. Select a single best model from $\mathcal{M}_0, \dots, \mathcal{M}_p$ using **cross-validated prediction error**, C_p , AIC, BIC, or adjusted R^2 .

Remark. RSS decreases monotonically and R^2 increases monotonically as k grows — they cannot be used to select among $\mathcal{M}_0, \dots, \mathcal{M}_p$ directly, as they always favour the largest model. Step 3 requires a criterion that accounts for model complexity.

16.2 Computational Limitation

The total number of models is 2^p . This grows rapidly with p :

- $p = 10$: $\approx 1,000$ models.
- $p = 20$: $> 1,000,000$ models.

Best subset selection is **computationally infeasible for large p** .

17 Forward Stepwise Selection

17.1 Algorithm

Forward stepwise selection is a **computationally efficient alternative** that considers a much smaller set of models. It starts with no predictors and adds them one at a time.

1. Let $\mathcal{M}_0$ denote the **null model** with no predictors.
2. For $k = 0, \dots, p - 1$:
 - (a) Consider all $p - k$ models that augment $\mathcal{M}_k$ with one additional predictor.
 - (b) Choose the best among these (smallest RSS or highest R^2) and call it $\mathcal{M}_{k+1}$.
3. Select a single best model from $\mathcal{M}_0, \dots, \mathcal{M}_p$ using cross-validation, C_p , AIC, BIC, or adjusted R^2 .

17.2 Computational Cost

Forward stepwise selection fits $1 + \sum_{k=0}^{p-1} (p - k) = 1 + p(p + 1)/2$ models in total. For $p = 20$: only **211 models** versus 1,048,576 for best subset selection.

Remark. Forward stepwise selection does **not** guarantee finding the globally best model among all 2^p subsets, since it follows a greedy path. However, it can be applied even when $n < p$, whereas best subset selection cannot.

18 Backward Stepwise Selection

18.1 Algorithm

Backward stepwise selection is an efficient alternative that begins with the **full model** and iteratively removes the least useful predictor.

1. Let $\mathcal{M}_p$ denote the **full model** with all p predictors.
2. For $k = p, p - 1, \dots, 1$:
 - (a) Consider all k models that contain all but one of the predictors in $\mathcal{M}_k$ (i.e., $k - 1$ predictors each).
 - (b) Choose the best (smallest RSS) and call it $\mathcal{M}_{k-1}$.
3. Select a single best model from $\mathcal{M}_0, \dots, \mathcal{M}_p$ using cross-validation, C_p , AIC, BIC, or adjusted R^2 .

Like forward stepwise, backward stepwise searches through only $1 + p(p + 1)/2$ models. It **requires** $n > p$ so that the full model can be fit. Forward stepwise can be used when $n < p$ and is therefore the only viable stepwise method in the high-dimensional setting.

18.2 Comparison of the Three Algorithms

The three algorithms generally give **similar but not identical** models. A **hybrid approach** also exists: variables are added sequentially (like forward selection) but any variable whose p -value rises above a threshold after new variables are added is removed (like backward selection), attempting to mimic best subset selection while retaining the computational advantages of stepwise methods.

19 Choosing the Optimal Model

All three subset selection procedures produce a sequence of models $\mathcal{M}_0, \dots, \mathcal{M}_p$. To select the best among them we need to estimate the **test error**. RSS and R^2 track training error and cannot be used directly. There are two approaches.

19.1 Approach 1: Adjusting the Training Error

The training MSE ($= \text{RSS}/n$) systematically underestimates the test MSE. Four criteria adjust for model size d (number of predictors).

19.1.1 Mallows's C_p

$$C_p = \frac{1}{n}(\text{RSS} + 2d\hat{\sigma}^2)$$

where $\hat{\sigma}^2$ is an estimate of $\text{Var}(\epsilon)$. The penalty $2d\hat{\sigma}^2$ increases with d , correcting for the tendency of training RSS to underestimate test error. We select the model with the **lowest** C_p .

19.1.2 AIC — Akaike Information Criterion

Defined for models fit by maximum likelihood. For a multivariate linear model with Gaussian errors, maximum likelihood and least squares are equivalent, giving:

$$\text{AIC} = \frac{1}{n\hat{\sigma}^2}(\text{RSS} + 2d\hat{\sigma}^2)$$

AIC and C_p are proportional in this setting and select the same model.

19.1.3 BIC — Bayesian Information Criterion

$$\text{BIC} = \frac{1}{n}(\text{RSS} + \log(n) d\hat{\sigma}^2)$$

BIC replaces the factor 2 in C_p with $\log(n)$. Since $\log(n) > 2$ for $n > 7$, BIC places a **heavier penalty** on models with many variables and tends to select **smaller models** than C_p . We select the model with the lowest BIC.

19.1.4 Adjusted R^2

Standard R^2 always increases as more variables are added. The adjusted R^2 corrects for this:

$$\text{Adjusted } R^2 = 1 - \frac{\text{RSS}/(n - d - 1)}{\text{TSS}/(n - 1)}$$

A large adjusted R^2 indicates a small test error. Unlike R^2 , adding an unnecessary variable will decrease adjusted R^2 because the decrease in RSS is small relative to the increase in d . We select the model with the **highest** adjusted R^2 .

19.2 Approach 2: Direct Estimation of Test Error

We can directly estimate the test error using the **validation set approach** or **cross-validation**. Plotting the validation set error or k -fold CV error as a function of the number of predictors d typically shows a clear minimum, identifying the optimal model size.

All four indirect criteria (C_p , AIC, BIC, adjusted R^2) and the direct CV approach tend to **agree** in practice, identifying the same optimal model. When they disagree, direct CV is generally preferred as it makes fewer assumptions about the error distribution.

Part V

Regularization

Subset selection methods (best subset, forward/backward stepwise) reduce model complexity by selecting a **subset of predictors**. An alternative strategy is to fit a model containing **all p predictors** but to **constrain** (regularize) the coefficient estimates, shrinking them toward zero. This can significantly **reduce variance** at the cost of a small increase in bias, often leading to better prediction accuracy. The two best-known shrinkage techniques are **ridge regression** and the **lasso**.

20 Shrinkage Methods

Shrinkage Methods

Shrinkage methods fit a model that contains all p predictors using a technique that **constrains** or **regularizes** the coefficient estimates, equivalently **shrinking** them toward zero. Shrinking the coefficient estimates can significantly **reduce their variance**.

21 Ridge Regression

21.1 Formulation

Recall that the ordinary least squares (OLS) estimates $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p$ minimize the residual sum of squares:

$$\text{RSS} = \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2$$

Ridge regression is very similar, but the coefficient estimates $\hat{\beta}_\lambda^R$ minimize a **penalized** objective:

Ridge Regression Objective

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 = \text{RSS} + \lambda \sum_{j=1}^p \beta_j^2$$

where $\lambda \geq 0$ is a **tuning parameter**.

Like OLS, ridge regression seeks coefficients that fit the data well by making the RSS small. However, the second term — the **shrinkage penalty** $\lambda \sum_j \beta_j^2$ (an ℓ_2 penalty) — is small when $\beta_1, \dots, \beta_p$ are close to zero.

21.2 The Tuning Parameter λ

The parameter λ controls the **relative impact** of the two terms:

- $\lambda = 0$: the penalty has no effect $\Rightarrow$ ridge regression coincides with **least squares**.
- $\lambda \rightarrow \infty$: the penalty dominates $\Rightarrow$ all coefficient estimates are driven toward zero.

Ridge regression produces a **different set of coefficient estimates** $\hat{\beta}_\lambda^R$ for each value of λ . As λ increases from zero, the ridge coefficient estimates shrink toward zero.

Remark. The shrinkage penalty is **not** applied to β_0 , which is simply the mean of the response when all predictors are centered. Only $\beta_1, \dots, \beta_p$ are penalized.

21.3 Scale Sensitivity and Standardization

Unlike least squares, ridge regression is **not scale invariant**: multiplying a predictor X_j by a constant c changes the ridge coefficient estimates because the penalty $\lambda \sum_j \beta_j^2$ depends on the scale of each predictor. Therefore, one should **standardize the predictors** before applying ridge regression:

$$\tilde{x}_{ij} = \frac{x_{ij} - \bar{x}_j}{\sigma_j} \quad \text{where} \quad \sigma_j = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_{ij} - \bar{x}_j)^2}$$

so that all predictors are on the same scale and the final fit does not depend on the units of measurement.

21.4 The ℓ_2 Norm Perspective

Let the ℓ_2 norm of the coefficient vector be $\|\beta\|_2 = \sqrt{\sum_{j=1}^p \beta_j^2}$. The ratio $\|\hat{\beta}_\lambda^R\|_2 / \|\hat{\beta}\|_2$ ranges from 1 (when $\lambda = 0$, recovering OLS) to 0 (when $\lambda = \infty$). This ratio measures the **amount by which** the ridge regression coefficients have been shrunk toward zero relative to the OLS solution.

21.5 Bias-Variance Trade-off in Ridge Regression

The advantage of ridge regression over least squares is rooted in the **bias-variance trade-off**:

- As λ increases, the **flexibility** of the ridge regression fit **decreases**: variance decreases but bias increases.
- For an intermediate value of λ , the MSE can be **considerably lower** than the OLS MSE, because the reduction in variance more than compensates for the increase in bias.

21.6 When to Use Ridge Regression

When Ridge Regression Helps

- When the relationship between response and predictors is close to **linear**, the OLS estimates have low bias but may have **high variance** — a small change in the training data can cause large changes in the coefficient estimates.
- In particular, when p is almost as large as n , OLS estimates are **extremely variable**.
- If $p > n$, OLS does not even have a **unique solution**, whereas ridge regression performs well by trading a small increase in bias for a large decrease in variance.
- Ridge regression has substantial **computational advantages** over best subset selection: fitting 2^p models vs. a single model for any value of λ .

Remark. With **multicollinearity** among predictors, the OLS loss surface is a long valley (a “ridge”), making the solution unstable. The ℓ_2 penalty turns this ridge into a **more regular** (better conditioned) function, stabilizing the estimates.

22 The Lasso

22.1 Motivation

Ridge regression has one important **disadvantage**: it includes **all p predictors** in the final model. The penalty shrinks all coefficients toward zero, but it does **not** set any of them **exactly to zero** (unless $\lambda = \infty$). This is not a problem for prediction accuracy, but it can hinder **model interpretability**, especially when p is large.

22.2 Formulation

The **Lasso** (Least Absolute Shrinkage and Selection Operator) overcomes this disadvantage. The lasso coefficient estimates $\hat{\beta}_\lambda^L$ minimize:

Lasso Objective

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p |\beta_j| = \text{RSS} + \lambda \sum_{j=1}^p |\beta_j|$$

The lasso uses an ℓ_1 penalty instead of the ℓ_2 penalty: $\|\beta\|_1 = \sum_{j=1}^p |\beta_j|$.

22.3 Variable Selection

The ℓ_1 penalty has the effect of **forcing some coefficient estimates to be exactly zero** when λ is sufficiently large. As a consequence, the lasso performs **variable selection**, yielding **sparse models** that involve only a subset of the predictors.

- $\lambda = 0$: the lasso gives the **OLS** fit.
- λ sufficiently large: the lasso gives the **null model** (all coefficients equal to zero).
- Between these extremes, the lasso produces models with **any number of variables**, depending on λ .

In contrast, ridge regression always includes all variables in the model (with magnitude depending on λ). Selecting a good value for the lasso parameter is critical — as with ridge regression, **cross-validation** is used to select the optimal λ .

23 Constrained Formulation

The lasso and ridge regression objectives can be equivalently written as **constrained optimization** problems:

Constrained Formulations**Lasso:**

$$\min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \right\} \quad \text{subject to} \quad \sum_{j=1}^p |\beta_j| \leq s$$

Ridge:

$$\min_{\beta} \left\{ \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \right\} \quad \text{subject to} \quad \sum_{j=1}^p \beta_j^2 \leq s$$

We seek coefficient estimates that make the RSS as small as possible, subject to the requirement that the ℓ_1 (lasso) or ℓ_2 (ridge) norm does not exceed a **budget** s . The budget s absorbs the role of λ : when s is extremely large, the constraint is not restrictive and the regression yields the OLS solution; when s is small, the coefficients must be small to avoid budget violation.

23.1 Geometric Interpretation

For $p = 2$, the constraint region of the lasso is a **diamond** ($|\beta_1| + |\beta_2| \leq s$), while the constraint region of ridge regression is a **disk** ($\beta_1^2 + \beta_2^2 \leq s$). The elliptical contours of the RSS expand outward from the OLS solution $\hat{\beta}$. The regularized solution is the point where the contours first **touch** the constraint region.

Because the lasso constraint has **corners** (at the axes), the contours are more likely to intersect the constraint at a corner, where one coefficient is exactly zero. The ridge constraint, being

smooth, has no corners, so the intersection almost never occurs at an axis — hence ridge rarely produces exact zeros.

24 Lasso vs Ridge Regression

Comparison

- **Lasso advantage:** produces simpler and more **interpretable** models that involve only a **subset of the predictors**, generating more accurate predictions when few predictors are truly related to the response.
- **Ridge advantage:** performs better when the response is a function of **many predictors**, all with coefficients of roughly equal size.
- The number of predictors related to the response is **never known a priori** in real problems.
- **Cross-validation** is used to determine the better approach on a particular dataset.

25 Selecting the Tuning Parameter

Both ridge regression and the lasso require a method for selecting the tuning parameter λ (or equivalently the budget s).

Tuning Parameter Selection via Cross-Validation

1. Choose a grid of λ values.
2. Compute the cross-validation error CV_λ for each λ .
3. Select the value of λ that minimizes the cross-validation error.
4. Re-fit the model using all available training data and the selected λ .

The cross-validation error as a function of λ is typically a U-shaped curve: too little regularization (small λ) leads to overfitting (high variance), while too much regularization (large λ) leads to underfitting (high bias). The optimal λ balances the two.

Part VI

Dimension Reduction

Subset selection and shrinkage control variance by working with a subset of predictors or by shrinking their coefficients. A complementary family of approaches instead **transforms the predictors** and then fits a model in the transformed space — this is the idea behind **dimension reduction**.

26 Dimension Reduction Methods

26.1 The General Framework

Let $Z_1, Z_2, \dots, Z_M$ denote M **linear combinations** of the original p predictors, where $M < p$:

$$Z_m = \sum_{j=1}^p \phi_{jm} X_j$$

for some constants $\phi_{1m}, \phi_{2m}, \dots, \phi_{pm}$ and $m = 1, \dots, M$. We then fit a linear regression model using least squares on these M transformed predictors:

$$y_i = \theta_0 + \sum_{m=1}^M \theta_m z_{im}, \quad i = 1, \dots, n.$$

26.2 Why Dimension Reduction?

The dimension of the regression problem is **reduced**: instead of estimating $p + 1$ coefficients $\beta_0, \beta_1, \dots, \beta_p$, we only need to estimate $M + 1$ coefficients $\theta_0, \theta_1, \dots, \theta_M$, where $M < p$.

Dimension reduction works in **two steps**:

1. The transformed predictors $Z_1, Z_2, \dots, Z_M$ are obtained.
2. A linear model is fit using these M predictors via least squares.

26.3 Connection to Linear Regression

The reduced model is a **special case** of the original linear regression model. Substituting the definition of Z_m , one can show:

$$\beta_j = \sum_{m=1}^M \theta_m \phi_{jm}$$

This imposes a **constraint** on the estimated β_j coefficients, which introduces a potential bias. However:

- When p is large relative to n , selecting $M \ll p$ can significantly **reduce the variance** of the fitted coefficients — a worthwhile bias-variance trade-off.
- When $M = p$ and all Z_m are linearly independent, no constraints are imposed and fitting the reduced system is **equivalent** to fitting the original model with coefficients β_j .

The choice of $Z_1, \dots, Z_M$ (i.e., the selection of ϕ_{jm}) can be achieved in different ways. The two approaches covered here are **principal components** and **partial least squares**.

27 Principal Components Analysis

27.1 Overview

Principal Components Analysis (PCA)

PCA is the process by which **principal components** are computed and used to understand the data. It derives a **low-dimensional set of features** from a large one, finding a small number of dimensions that are as **interesting** as possible — where “interesting” is measured by the amount that the observations **vary** along each dimension.

Key properties of PCA:

- PCA is an **unsupervised approach**: it involves only the features $X_1, X_2, \dots, X_p$ and no associated response Y .
- It is primarily a tool for **data visualization**, but is also useful in **supervised learning** (via Principal Components Regression).
- Each dimension found by PCA is a **linear combination** of the p features.

27.2 Principal Component Directions

Consider n observations with measurements on p features. Each observation lives in a p -dimensional space, but **not all dimensions are equally interesting**. PCA seeks directions along which the data are most variable.

Principal Component Directions

Principal component directions are the directions in feature space along which the original data are **most variable**.

Example: population size and ad spending for 100 cities. The **first principal component direction** (green solid line) captures the direction of maximum variance. The **second principal component direction** (blue dashed line) is perpendicular to the first and captures the remaining variance.

27.3 The First Principal Component

The **first principal component** of a set of p features is the normalized linear combination of the features:

$$Z_1 = \phi_{11}X_1 + \phi_{21}X_2 + \cdots + \phi_{p1}X_p$$

with the **largest variance**, subject to the normalization constraint $\sum_{j=1}^p \phi_{j1}^2 = 1$.

The elements $\phi_{11}, \dots, \phi_{p1}$ are the **loadings** of the first principal component; together they form the **loading vector** $\phi_1 = (\phi_{11}, \phi_{21}, \dots, \phi_{p1})^T$.

27.3.1 Optimization Problem

We solve:

$$\max_{\phi_{11}, \dots, \phi_{p1}} \left\{ \frac{1}{n} \sum_{i=1}^n \left(\sum_{j=1}^p \phi_{j1} x_{ij} \right)^2 \right\} \quad \text{subject to} \quad \sum_{j=1}^p \phi_{j1}^2 = 1$$

The values $z_{11}, \dots, z_{n1}$ are known as the **principal component scores**: $z_{i1} = \sum_{j=1}^p \phi_{j1} x_{ij}$.

27.3.2 Geometric Interpretation

The loading vector ϕ_1 defines the line in p -dimensional feature space that is **as close as possible to the data**, in the sense that it **minimizes the sum of squared perpendicular distances** between each point and the line. Projecting observations onto this line yields principal component scores.

Example (advertising data): the first principal component is:

$$Z_1 = 0.839 \times (pop - \overline{pop}) + 0.544 \times (ad - \overline{ad})$$

where $\phi_{11} = 0.839$ and $\phi_{21} = 0.544$ are the loadings; $\overline{pop}$ and $\overline{ad}$ are sample means (features are centered before computing PCA).

27.4 The Second Principal Component

We can construct up to p distinct principal components. The **second principal component** Z_2 is the linear combination of the variables that:

- has **largest variance** subject to being **uncorrelated with** Z_1 ,

- must be **perpendicular** (orthogonal) to the first principal component direction.

For the advertising data (with $p = 2$):

$$Z_2 = 0.544 \times (pop - \overline{pop}) + 0.839 \times (ad - \overline{ad})$$

Since $p = 2$, the two principal components together contain **all of the information** in pop and ad .

Key Observation

The second principal component scores z_{i2} are much closer to zero than z_{i1} , indicating that the second component captures **less information**. Plots of z_{i2} versus pop and ad show weak relationships: only the **first principal component is needed** to accurately represent both predictors.

Additional components, if more predictors were available, would **successively maximize variance** subject to the constraint of being uncorrelated with all preceding components.

28 Principal Components Regression

28.1 The PCR Approach

Principal Components Regression (PCR)

PCR constructs the first M principal components $Z_1, \dots, Z_M$, and then uses them as predictors in a **linear regression model fit by least squares**.

Key idea: often a small number of principal components suffice to explain most of the variability in the data, as well as the relationship with the response.

Core assumption: the directions that show the most variation in X are also the most **associated with Y** . This assumption is not guaranteed to be true, but it is often reasonable and provides good empirical results.

28.2 Advantages of PCR

If the assumption above holds:

- Fitting least squares to $Z_1, \dots, Z_M$ yields **better results** than fitting to $X_1, \dots, X_p$, since most or all of the information relevant to the response is contained in $Z_1, \dots, Z_M$.
- By estimating only $M \ll p$ coefficients we can **mitigate overfitting**.

Example: for the advertising data, the first principal component explains most of the variance in both pop and ad . A PCR model using this single variable to predict $sales$ performs well.

28.3 Choosing M via Cross-Validation

In PCR, the number of principal components M is a **tuning parameter** typically chosen by **cross-validation**. The cross-validation MSE as a function of M is examined; the optimal M minimizes this error.

28.4 PCR is Not Feature Selection

Important: PCR $\neq$ Feature Selection

Even though PCR provides a simple way to perform regression using $M < p$ predictors, it is **not a feature selection method**. Each of the M principal components used in the regression is a **linear combination of all p original features**.

For instance, the first principal component Z_1 was a linear combination of both *pop* and *ad*. In this sense, PCR is more closely related to **ridge regression** than to the lasso. Ridge regression can in fact be considered as a **continuous version of PCR**.

28.5 PCR vs Ridge Regression

Both PCR and ridge regression perform well on data where the first few principal components capture most of the variation relevant to the response. Empirically:

- PCR shows a clear minimum in test MSE at some $M^* < p$, with bias dropping rapidly as M increases.
- Ridge regression provides a smooth, continuous regularization; PCR provides a discrete one (by choosing how many components to include).
- On data designed to depend on the first five principal components, both PCR (at optimal M) and ridge outperform the lasso.

29 Partial Least Squares

29.1 Motivation: The Supervised Alternative to PCA

PCR identifies directions that best represent the predictors $X_1, \dots, X_p$ in an **unsupervised** way: the response Y plays no role in determining the principal component directions.

Main Drawback of PCR

There is no guarantee that the directions that best explain the predictors will also be the best directions for **predicting the response**.

Partial Least Squares (PLS)

PLS is a **supervised** dimension reduction method. It identifies a new set of features $Z_1, \dots, Z_M$ that are linear combinations of the original features, and then fits a linear model via least squares using these M new features. Unlike PCR, PLS identifies the new features in a **supervised** way — making use of Y to find directions that explain both the predictors and the response.

29.2 Computing PLS Directions

First PLS direction Z_1 : after standardizing the p predictors, PLS computes the first direction by setting each loading ϕ_{j1} equal to the **coefficient from the simple linear regression of Y onto X_j** . This coefficient is proportional to the correlation between Y and X_j . Hence:

$$Z_1 = \sum_{j=1}^p \phi_{j1} X_j$$

places the **highest weight** on the variables most strongly correlated with the response.

Example: on the advertising data, the first PLS direction has less change in the *ad* dimension per unit change in the *pop* dimension, relative to PCA. This suggests that *pop* is more highly correlated with the response than *ad*.

Second PLS direction Z_2 : we first **adjust** each variable for Z_1 , by regressing each variable on Z_1 and taking the **residuals** (the information not explained by the first PLS direction). We then compute Z_2 from this orthogonalized data in the same fashion as Z_1 .

This iterative procedure can be repeated up to M times to identify PLS components $Z_1, \dots, Z_M$.

29.3 Fitting and Tuning

At the end of the procedure, least squares is used to fit a linear model to predict Y using $Z_1, \dots, Z_M$. As with PCR, the **number M of PLS directions is a tuning parameter** typically chosen by **cross-validation**.

Remark. While PLS can reduce bias relative to PCR (since it uses Y to find directions), it can also **increase variance**. In practice, PLS often performs no better than ridge regression or PCR. The theoretical properties of PLS are less well understood than those of PCR.

30 Considerations in High Dimensions

30.1 Low-Dimensional vs High-Dimensional Data

Most traditional statistical techniques for regression and classification are intended for the **low-dimensional setting** in which $n \gg p$ (many more observations than features).

Example: predicting a patient's blood pressure from age, gender, and BMI. Here $p = 3$ (or 4 with an intercept) and n could be several thousand patients — clearly $n \gg p$, a low-dimensional problem.

In the past 20 years, new technologies have enabled collection of an almost unlimited number of feature measurements (p very large), while n may be limited by cost or sample availability. Data sets with **more features than observations** ($p > n$) are called **high-dimensional**.

Examples:

- Predicting blood pressure from half a million SNPs (single nucleotide polymorphisms): $n \approx 200$, $p \approx 500,000$.
- Online shopping pattern analysis using the **bag-of-words** model: each search term is a binary feature; p can easily exceed n .

30.2 What Goes Wrong in High Dimensions?

Classical approaches such as **least squares linear regression are not appropriate** in the high-dimensional setting. When $p \geq n$, least squares will yield coefficient estimates that result in a **perfect fit to the training data** (residuals are exactly zero) — regardless of whether there truly is a relationship between the features and the response.

The High-Dimensional Danger

When $p \geq n$, least squares always produces a perfect fit to training data. This perfect fit **almost certainly leads to overfitting**: the model performs extremely poorly on an independent test set, since it is too flexible and captures noise.

Illustration: with $n = 20$ observations and $p = 1$ predictor, least squares does not perfectly fit the data. With only $n = 2$ observations, any line passes through both points — a perfect fit that generalizes nothing.

30.3 The Curse of Dimensionality in Practice

The principal issues in high-dimensional analysis are:

- The **bias-variance trade-off** becomes critical: adding more features can increase variance dramatically.
- The danger of **overfitting** is amplified when p is large relative to n .
- These considerations also apply when p is only **slightly smaller** than n .

Empirically: simulating $n = 20$ observations with regression on p varying from 1 to 20 (all features unrelated to the response):

- R^2 **increases to 1** as p increases — even though no feature is relevant.
- Training MSE **decreases to 0** as p increases.
- Test MSE **increases dramatically** as p increases.

Including additional predictors leads to a **huge increase in the variance of the coefficient estimates**. This highlights the importance of always evaluating model performance on an **independent test set**, and of **never relying solely on R^2 or training MSE** as measures of model quality.

30.4 Implications for Model Selection Criteria

Approaches for adjusting training RSS or R^2 to account for model size — such as C_p , AIC, and BIC — are **not appropriate in the high-dimensional setting**, because estimating $\hat{\sigma}^2$ is problematic when $p \geq n$ (the residual degrees of freedom become zero or negative).

High-Dimensional Takeaways

- Regularization and dimension reduction methods (ridge, lasso, PCR, PLS) are essential in high dimensions.
- Always report performance on an **independent test set** or via **cross-validation**.
- Be cautious of R^2 and training MSE — they are unreliable indicators of model quality when p is large.
- The best model in high dimensions contains **only a small number of relevant features**: use regularization to enforce sparsity.

Part VII

Classification

Classification is the task of predicting a **qualitative** (categorical) response. Just as regression is used for quantitative outputs, classification methods are used when the output takes on one of K discrete classes. Many of the core ideas from regression — bias-variance trade-off, training vs. test error, model selection — carry over with only minor modifications.

31 Introduction to Classification

31.1 Qualitative Variables

In many situations, the response variable is **qualitative**: it takes values from a finite, unordered set of categories (e.g., eye color: blue, brown, green). The process of predicting a qualitative response is known as **classification** — it involves **assigning the observation to a category, or class**.

Classifiers typically work by first **estimating the probability** that an observation belongs to each possible class, and then making a decision based on those probabilities. Three of the most widely used classifiers are:

- **K-Nearest Neighbours (KNN)**
- **Logistic Regression**
- **Linear Discriminant Analysis (LDA)**

Example applications:

1. A patient arrives at the emergency room with a set of symptoms — classify into one of three medical conditions.
2. Fraud detection: determine whether a transaction is fraudulent, based on IP address, transaction history, etc.
3. Predict, on the basis of DNA sequence data, which mutations are disease-causing.

31.2 The Classification Setting

We seek to estimate the classification function f from training observations $\{(x_1, y_1), \dots, (x_n, y_n)\}$ where $y_1, \dots, y_n$ are qualitative. The accuracy of $\hat{f}$ is quantified by the **training error rate**:

$$\varepsilon_{\text{train}} = \frac{1}{n} \sum_{i=1}^n I(y_i \neq \hat{y}_i)$$

where $\hat{y}_i$ is the predicted class label and $I(\cdot)$ is the indicator function (1 if $y_i \neq \hat{y}_i$, zero otherwise). $\varepsilon_{\text{train}}$ is simply the fraction of incorrect classifications on the training data. The **test error rate** is computed from predictions on data not used for training — this is what we actually care about.

31.3 The Bayes Classifier

The Bayes Classifier

The test error rate is minimized, on average, by a classifier that assigns each observation to the **most likely class given its predictor values**. That is, assign a test observation with predictor vector x_0 to the class j for which

$$\Pr(Y = j \mid X = x_0)$$

is the **largest**. This is the **Bayes classifier**.

In a **two-class problem** (classes 1 and 2), the Bayes classifier assigns an observation to class 1 if $\Pr(Y = 1 \mid X = x_0) > 0.5$, and to class 2 otherwise.

31.3.1 Bayes Decision Boundary

The **Bayes decision boundary** is the set of points where the largest conditional class probability is exactly 0.5 (in the two-class case). Observations on one side of the boundary are assigned to one class, and observations on the other side to the other class.

The Bayes classifier produces the **lowest possible test error rate**, called the **Bayes error rate**. At a given $X = x_0$, the error rate is $1 - \max_j \Pr(Y = j \mid X = x_0)$. The overall Bayes error rate is:

$$1 - \mathbb{E} \left[\max_j \Pr(Y = j \mid X) \right]$$

where the expectation averages over all possible values of X .

The Bayes Classifier is Unattainable

For real data, we do **not** know the conditional distribution of Y given X , so computing the Bayes classifier is **impossible**. It serves as an **unattainable gold standard** against which to compare other methods.

32 K-Nearest Neighbours

32.1 The KNN Classifier

Many methods estimate the conditional distribution of Y given X , and then classify an observation to the class with the highest estimated probability. One of the most widely used is the **K-Nearest Neighbours (KNN)** classifier.

KNN Algorithm

Given a positive integer K and a test observation x_0 , KNN:

1. Identifies the K training points closest to x_0 — call this set $\mathcal{N}_0$.
2. Estimates the conditional probability for class j as the **fraction of points in $\mathcal{N}_0$ whose response equals j** :

$$\Pr(Y = j \mid X = x_0) = \frac{1}{K} \sum_{i \in \mathcal{N}_0} I(y_i = j)$$

3. Classifies x_0 to the class with the **largest probability**.

Example with $K = 3$: to predict the class of a test point (black cross), KNN finds the three closest training observations — suppose two are blue and one is orange. The estimated probabilities are $2/3$ for blue and $1/3$ for orange. KNN predicts the **blue class**.

32.2 Choice of K : Bias-Variance Trade-off

The value of K is the key tuning parameter and controls the **flexibility** of the classifier:

- $K = 1$: the decision boundary is **overly flexible**, fits patterns not corresponding to the Bayes boundary — **low bias, very high variance**. Training error rate is exactly 0.
- **Large K** (e.g., $K = 100$): the boundary becomes **less flexible**, close to linear — **lower variance, higher bias**.
- **Optimal K** (e.g., $K = 10$ on a simulated dataset): KNN decision boundary is close to the Bayes boundary; test error rate (0.1363) is close to the Bayes error rate (0.1304).

Flexibility and Error in KNN

As $1/K$ increases (more flexible), training error consistently decreases. However, the test error forms a characteristic **U-shape**: too flexible (small K) leads to overfitting and high test error; too rigid (large K) leads to underfitting. The optimal K minimizes test error.

Part VIII

Classification II — Logistic Regression

33 Why Not Linear Regression?

33.1 The Problem with Encoding Qualitative Responses

A natural first attempt at classification is to encode the classes as numbers and apply linear regression. However, this approach has a fundamental flaw: the **order of the encoding matters**.

Example: predict a medical condition — stroke, drug overdose, epileptic seizure. We could encode:

$$Y = \begin{cases} 1 & \text{if stroke} \\ 2 & \text{if drug overdose} \\ 3 & \text{if epileptic seizure} \end{cases}$$

and use least squares to fit a linear model on the predictors $X_1, \dots, X_p$. But this encoding **implies an ordering and equal spacing** between conditions that does not exist medically. A different encoding (e.g., stroke = 1, epileptic seizure = 2, drug overdose = 3) would produce **fundamentally different linear models** and hence different predictions.

The only situation where a quantitative encoding is defensible is when the response classes have a **natural ordering and equal gaps** — for instance:

$$Y = \begin{cases} 1 & \text{if mild} \\ 2 & \text{if moderate} \\ 3 & \text{if severe} \end{cases}$$

In general, there is **no natural way** to encode a qualitative response with more than two values into a single quantitative response.

33.2 The Binary Case: Linear Regression on a Dummy Variable

When the response has only **two classes**, we can use a binary (dummy variable) encoding:

$$Y = \begin{cases} 0 & \text{if stroke} \\ 1 & \text{if drug overdose} \end{cases}$$

and fit a linear regression model to predict $\Pr(\text{drug overdose} \mid X) \approx X\hat{\beta}$. Then classify as drug overdose if $\hat{Y} > 0.5$, and stroke otherwise.

This works for two classes, but has two critical issues:

1. The linear regression estimate $X\hat{\beta}$ can fall **outside the interval** $[0, 1]$ — giving negative probabilities or probabilities above 1, which are not valid.

2. The approach **cannot be extended** to qualitative responses with more than two values.

Conclusion

We need classification methods that are specifically designed for qualitative responses and that always produce **valid probability estimates** in $[0, 1]$.

34 Logistic Regression

34.1 The Logistic Model

Rather than modelling Y directly, logistic regression models the **probability that Y belongs to a particular category**. For the Default dataset — response: default (Yes/No), predictor: credit card balance — we model:

$$p(\text{balance}) = \Pr(\text{default} = \text{Yes} \mid \text{balance})$$

Using a linear model $p(X) = \beta_0 + \beta_1 X$ leads to estimates outside $[0, 1]$ for extreme values of X . To avoid this, logistic regression uses the **logistic function**:

The Logistic Function

$$p(X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}$$

The logistic function always produces outputs in $(0, 1)$ for any value of X , giving an **S-shaped curve** — always a reasonable probability prediction. The model is fit using **maximum likelihood**.

Illustration: for the Default data, linear regression produces some **negative** estimated probabilities of default. Logistic regression keeps all probabilities in $[0, 1]$.

34.2 The Odds

After algebraic manipulation of the logistic function, we obtain:

$$\frac{p(X)}{1 - p(X)} = e^{\beta_0 + \beta_1 X}$$

The quantity $p(X)/(1 - p(X))$ is called the **odds**, and ranges in $[0, \infty)$:

- Odds close to 0: very low probability of the event.
- Odds much larger than 1: very high probability.
- *Example:* $p(X) = 0.2 \Rightarrow \text{odds} = 0.2/0.8 = 1/4$ (on average 1 in 5 people will default).
- *Example:* $p(X) = 0.9 \Rightarrow \text{odds} = 0.9/0.1 = 9$ (9 out of 10 will default).

34.3 The Logit (Log-Odds)

Taking the **logarithm** of both sides:

$$\log\left(\frac{p(X)}{1 - p(X)}\right) = \beta_0 + \beta_1 X$$

The left-hand side is called the **logit** or **log-odds**. Key properties:

- The logit is **linear in X** : the logistic regression model is linear on the log-odds scale.
- Increasing X by one unit changes the log odds by β_1 , equivalently multiplying the odds by e^{β_1} .
- The change in $p(X)$ per unit increase in X **depends on the current value of X** (non-constant, unlike linear regression).
- If $\beta_1 > 0$: increasing X **increases** $p(X)$. If $\beta_1 < 0$: increasing X **decreases** $p(X)$.

34.4 Estimating the Coefficients

The coefficients β_0 and β_1 are estimated from training data. We could use (non-linear) least squares, but **maximum likelihood** is preferred as it has better statistical properties.

The Likelihood Function

We seek $\hat{\beta}_0$ and $\hat{\beta}_1$ such that predicted probabilities are **close to 1** for individuals who defaulted and **close to 0** for those who did not. This is formalized by the likelihood function:

$$\ell(\beta_0, \beta_1) = \prod_{i: y_i=1} p(x_i) \prod_{i': y_{i'}=0} (1 - p(x_{i'}))$$

The estimates $\hat{\beta}_0$ and $\hat{\beta}_1$ are chosen to **maximize** $\ell(\beta_0, \beta_1)$.

Remark. Least squares in the linear regression setting is a special case of maximum likelihood. Logistic regression and other models can typically be fit using a standard statistical software package.

34.4.1 Coefficient Estimates for the Default Dataset

Fitting logistic regression on the Default data to predict $\Pr(\text{default} = \text{Yes} \mid \text{balance})$:

	Coefficient	Std. error	Z-statistic	P-value
Intercept	-10.6513	0.3612	-29.5	< 0.0001
balance	0.0055	0.0002	24.9	< 0.0001

$\hat{\beta}_1 = 0.0055$: a one-unit increase in balance increases the **log odds of default by 0.0055**.

Many aspects of logistic regression are **analogous to linear regression**:

- The **z-statistic** plays the role of the t-statistic: $z = \hat{\beta}_1 / \text{SE}(\hat{\beta}_1)$. A large absolute value is evidence against $H_0 : \beta_1 = 0$.
- The **p-value** tests whether there is a statistically significant association between balance and default.

34.4.2 Making Predictions

Once coefficients are estimated, computing probabilities is straightforward. For a balance of \$1000:

$$\hat{p}(X) = \frac{e^{-10.6513 + 0.0055 \times 1000}}{1 + e^{-10.6513 + 0.0055 \times 1000}} \approx 0.00576$$

which is below 1%. For a balance of \$2000 the predicted probability is $0.586 = 58.6\%$ — much higher.

34.4.3 Using Qualitative Predictors

Qualitative predictors are included via **dummy variables**. For the Default dataset, student status is encoded as 0 (non-student) and 1 (student):

	Coefficient	Std. error	z-statistic	p-value
Intercept	−3.5041	0.0707	−49.55	< 0.0001
student[Yes]	0.4049	0.1150	3.52	0.0004

The **positive coefficient** for student[Yes] indicates that students have a **higher chance of default** than non-students (when using student status as the only predictor).

35 Multiple Logistic Regression

35.1 Extending to Multiple Predictors

By analogy with the extension from simple to multiple linear regression, the logistic model generalizes to **multiple predictors** $X = (X_1, \dots, X_p)$:

Multiple Logistic Regression

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p$$

equivalently:

$$p(X) = \frac{e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}{1 + e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}$$

Parameters $\beta_0, \beta_1, \dots, \beta_p$ are estimated via **maximum likelihood**.

35.2 Example: Default Dataset with Three Predictors

Fitting logistic regression using balance, income, and student status simultaneously:

	Coefficient	Std. error	z-statistic	p-value
Intercept	−10.8690	0.4923	−22.08	< 0.0001
balance	0.0057	0.0002	24.74	< 0.0001
income	0.0030	0.0082	0.37	0.7115
student[Yes]	−0.6468	0.2362	−2.74	0.0062

Confounding: the Student Sign Reversal

When using student status alone, the coefficient is **positive** (+0.4049): students appear to default more. When controlling for balance and income, the coefficient becomes **negative** (−0.6468): students actually default *less* at a fixed balance level.

This is a case of **confounding**: students tend to carry higher credit card balances, and high balance is the true driver of default. Once balance is held fixed, being a student is associated with **lower** default risk. This illustrates the importance of considering all relevant predictors simultaneously — a single-predictor model can give a misleading picture.

Part IX

Classification III — Linear Discriminant Analysis

36 Introduction

36.1 An Alternative Approach to Classification

Logistic regression models $\Pr(Y = k \mid X = x)$ **directly** using the logistic function. Linear Discriminant Analysis (LDA) takes a different, **less direct** approach:

1. Model the distribution of the predictors X **separately within each class** — i.e., estimate $f_k(x) = \Pr(X = x \mid Y = k)$ for each class k .
2. Use **Bayes' theorem** to flip these around into estimates of $\Pr(Y = k \mid X = x)$.

When the within-class distributions are assumed to be **Gaussian**, the resulting model turns out to be very similar to logistic regression.

36.2 Why Use LDA?

Advantages of LDA over Logistic Regression

- When the classes are **well-separated**, parameter estimates for logistic regression are surprisingly **unstable**. LDA does not suffer from this problem.
- When n is **small** and the distribution of X is approximately **normal** within each class, LDA is more stable than logistic regression.
- LDA is naturally suited for problems with **more than two response classes**.

37 Bayes' Theorem for Classification

37.1 Setup

Suppose we wish to classify an observation into one of K classes, where $K \geq 2$ — the categorical variable Y takes K possible distinct and unordered values. Define:

- $\pi_k =$ **prior probability** that a randomly chosen observation comes from class k ; this is the probability that a given observation belongs to the k -th category of Y .
- $f_k(x) = \Pr(X = x \mid Y = k) =$ **density function** of X for an observation from class k . $f_k(x)$ is large if it is highly probable that an observation in class k has $X \approx x$, and small if it is very unlikely.

37.2 Bayes' Theorem

Bayes' Theorem for Classification

$$p_k(x) = \Pr(Y = k \mid X = x) = \frac{\pi_k f_k(x)}{\sum_{j=1}^K \pi_j f_j(x)}$$

$p_k(x)$ is the **posterior probability** that an observation $X = x$ belongs to class k — i.e., the probability that the observation belongs to the k -th class, given the predictor value.

- Estimating π_k is **easy**: compute the **fraction of training observations** that belong to class k .
- Estimating $f_k(x)$ is more **challenging** — we need to assume some form for these densities. LDA assumes they are Gaussian.

The Bayes classifier, which achieves the lowest possible error rate, assigns $X = x$ to the class with the largest $p_k(x)$. If we can estimate $f_k(x)$, we can build a classifier that **approximates the Bayes classifier**.

38 Linear Discriminant Analysis for $p = 1$

38.1 Gaussian Assumption

With a single predictor ($p = 1$), assume $f_k(x)$ is **normal (Gaussian)**:

$$f_k(x) = \frac{1}{\sqrt{2\pi} \sigma_k} \exp\left(-\frac{1}{2\sigma_k^2}(x - \mu_k)^2\right)$$

where μ_k and σ_k^2 are the mean and variance for class k .

LDA further assumes a **shared variance** across all K classes: $\sigma_1^2 = \dots = \sigma_K^2 = \sigma^2$.

Plugging into the Bayes formula:

$$p_k(x) = \frac{\pi_k \frac{1}{\sqrt{2\pi} \sigma} \exp\left(-\frac{1}{2\sigma^2}(x - \mu_k)^2\right)}{\sum_{j=1}^K \pi_j \frac{1}{\sqrt{2\pi} \sigma} \exp\left(-\frac{1}{2\sigma^2}(x - \mu_j)^2\right)}$$

The Bayes classifier assigns $X = x$ to the class for which this expression is largest.

38.2 Discriminant Functions

Taking the logarithm and rearranging, maximizing $p_k(x)$ over k is equivalent to assigning x to the class with the largest **linear discriminant function**:

Linear Discriminant Function ($p = 1$)

$$\delta_k(x) = x \cdot \frac{\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} + \log(\pi_k)$$

These functions are **linear in x** — hence the name *Linear* Discriminant Analysis.

Example ($K = 2$, $\pi_1 = \pi_2$): the Bayes classifier assigns x to class 1 if $2x(\mu_1 - \mu_2) > \mu_1^2 - \mu_2^2$, and to class 2 otherwise. The **Bayes decision boundary** is the point where $\delta_1(x) = \delta_2(x)$:

$$x = \frac{\mu_1^2 - \mu_2^2}{2(\mu_1 - \mu_2)} = \frac{\mu_1 + \mu_2}{2}$$

38.3 The LDA Algorithm

In practice, μ_k , π_k , and σ^2 are unknown. LDA estimates them from training data:

LDA Parameter Estimates

$$\hat{\mu}_k = \frac{1}{n_k} \sum_{i: y_i=k} x_i, \quad \hat{\sigma}^2 = \frac{1}{n-K} \sum_{k=1}^K \sum_{i: y_i=k} (x_i - \hat{\mu}_k)^2, \quad \hat{\pi}_k = \frac{n_k}{n}$$

where n is the total number of training observations and n_k is the number in class k .

- $\hat{\mu}_k$: average of all training observations from class k .
- $\hat{\sigma}^2$: weighted average of the sample variances for each class.
- $\hat{\pi}_k$: proportion of training observations in class k (or the known prior, if available).

LDA plugs these estimates into $\delta_k(x)$ and assigns $X = x$ to the class for which:

$$\hat{\delta}_k(x) = x \cdot \frac{\hat{\mu}_k}{\hat{\sigma}^2} - \frac{\hat{\mu}_k^2}{2\hat{\sigma}^2} + \log(\hat{\pi}_k)$$

is the largest.

38.4 Decision Boundary and Performance

Example: 20 observations drawn from each of two Gaussian classes ($\mu_1 = -1.25$, $\mu_2 = 1.25$, $\sigma^2 = 1$). The two density functions overlap, so there is inherent uncertainty about class membership.

Since $n_1 = n_2 = 20$, we have $\hat{\pi}_1 = \hat{\pi}_2$. The **LDA decision boundary** is the midpoint of the sample means:

$$\frac{\hat{\mu}_1 + \hat{\mu}_2}{2}$$

The **Bayes decision boundary** (optimal) is at $(\mu_1 + \mu_2)/2 = 0$. The LDA boundary (solid line) is close to but not exactly at the Bayes boundary (dashed line), due to estimation from finite data.

LDA Performance

- Bayes error rate: **10.6%** (theoretical minimum).
 - LDA test error rate: **11.1%** — only 0.5% above optimal.
- LDA closely approximates the Bayes classifier when the Gaussian assumption is satisfied.

39 Linear Discriminant Analysis for $p > 1$

39.1 Multivariate Gaussian Distribution

For multiple predictors, LDA assumes $X = (X_1, \dots, X_p)$ follows a **multivariate Gaussian distribution** $X \sim \mathcal{N}(\mu, \Sigma)$:

- Each individual predictor follows a one-dimensional normal distribution, with some **correlation** between each pair of predictors.
- $E(X) = \mu$ is a vector with p components (the mean of X).
- $\text{Cov}(X) = \Sigma$ is the $p \times p$ **covariance matrix** of X .

The multivariate Gaussian density is:

$$f(x) = \frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right)$$

LDA assumes each class k has its own **class-specific mean vector** μ_k , but all classes **share a common covariance matrix** Σ .

39.2 Discriminant Functions for $p > 1$

Plugging in the multivariate Gaussian density, the Bayes classifier assigns $X = x$ to the class for which the **multivariate linear discriminant function** is largest:

Linear Discriminant Function ($p > 1$)

$$\delta_k(x) = x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log \pi_k$$

This is the vector/matrix generalization of the univariate formula. Note that $\delta_k(x)$ is a **linear function of x** : the LDA decision rule depends on x only through a **linear combination** of its elements.

LDA estimates the unknown parameters $\mu_1, \dots, \mu_K$, $\pi_1, \dots, \pi_K$, and Σ from training data, plugs them into $\delta_k(x)$, and classifies to the class for which $\hat{\delta}_k(x)$ is the largest.

39.3 Example: Two Predictors, Three Classes

Three equally-sized Gaussian classes with class-specific mean vectors and a common covariance matrix. The **Bayes decision boundaries** (dashed lines) are defined by the set of values x for which $\delta_k(x) = \delta_l(x)$, i.e.:

$$x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k = x^T \Sigma^{-1} \mu_l - \frac{1}{2} \mu_l^T \Sigma^{-1} \mu_l \quad \text{for } k \neq l$$

With $K = 3$ classes there are three pairs, hence **three decision boundaries** dividing the predictor space into three regions. The Bayes classifier assigns an observation to the region in which it is located.

With 20 observations drawn from each class, the **LDA decision boundaries** (solid lines) are close to the Bayes decision boundaries (dashed lines):

- Bayes test error rate: **7.46%**
- LDA test error rate: **7.70%**

LDA performs well on this data.

40 Evaluating Classification Performance

40.1 Beyond the Overall Error Rate

A low overall error rate does not tell the full story. Consider LDA applied to the **Default dataset** (10,000 training samples, binary response: default Yes/No):

- LDA training error rate: **2.75%** — appears excellent.
- But the **null classifier** (always predicts “No default”) achieves **3.33%** — barely worse.

- Only 3.33% of individuals actually defaulted, so a useless classifier is almost as good by the overall error rate metric alone.

A binary classifier can make two types of error:

- **False Negative (FN)**: a true defaulter incorrectly assigned to the “No default” class.
- **False Positive (FP)**: a non-defaulter incorrectly assigned to the “Default” class.

40.2 The Confusion Matrix

A **confusion matrix** displays the counts of all four prediction outcomes:

		True default status		
		No	Yes	Total
Predicted default status	No	9 644	252	9 896
	Yes	23	81	104
Total		9 667	333	10 000

Reading the matrix: LDA predicted 104 people would default. Of these, 81 actually defaulted and 23 did not. Of the 333 true defaulters, **252 (75.7%) were missed** by LDA — a very high miss rate.

40.3 Key Metrics

Name	Definition	Synonyms
False Positive rate	FP / N	Type I error, 1 – Specificity
True Positive rate	TP / P	1 – Type II error, Sensitivity, Recall
Positive Pred. value	TP / P^*	Precision
Negative Pred. value	TN / N^*	—

For the Default example:

- **Sensitivity** (true defaulters correctly identified): $81/333 = 24.3\%$ — very low.
- **Specificity** (non-defaulters correctly identified): $9,644/9,667 = 99.8\%$ — very high.

Why Does LDA Miss So Many Defaulters?

LDA approximates the Bayes classifier, which minimises the **total** error rate irrespective of which class the errors come from. Since only 3.33% of observations default, the Bayes classifier sacrifices sensitivity on the minority class (defaulters) to keep the overall error low. A credit card company, however, may specifically want to **avoid missing defaulters**.

40.4 Adjusting the Decision Threshold

The Bayes classifier (and by extension LDA) uses a default threshold of **50%**:

$$\Pr(\text{default} = \text{yes} \mid X = x) > 0.5 \implies \text{predict Default}$$

To **increase sensitivity**, we can **lower the threshold**. Using 20%:

$$\Pr(\text{default} = \text{yes} \mid X = x) > 0.2 \implies \text{predict Default}$$

		True default status		
		No	Yes	Total
Predicted default status	No	9 432	138	9 570
	Yes	235	195	430
Total		9 667	333	10 000

With a 20% threshold: sensitivity rises to $195/333 = 58.6\%$ (vs. 24.3% before), but 235 non-defaulters are now incorrectly classified and the overall error rate increases slightly to 3.73%.

Threshold Trade-off

Lowering the threshold **increases sensitivity** but **decreases specificity**. The optimal threshold depends on the **domain costs** associated with each error type and cannot be determined from the data alone — it requires external domain knowledge (e.g., the financial cost of a missed default vs. a false alarm).

40.5 The ROC Curve and AUC

Rather than evaluating at a single threshold, the **ROC curve** (Receiver Operating Characteristics) simultaneously displays both error types **for all possible thresholds**:

- y -axis: **True Positive Rate** (sensitivity).
- x -axis: **False Positive Rate** ($1 - \text{specificity}$).

The **Area Under the ROC Curve (AUC)** summarises the overall classifier performance across all thresholds:

- $\text{AUC} = 1$: perfect classifier.
- $\text{AUC} = 0.5$: no better than random chance (diagonal dotted line).
- The higher the AUC, the better the classifier.

For the LDA classifier on the Default data, $\text{AUC} = \mathbf{0.95}$ — excellent overall discrimination between defaulters and non-defaulters.

Key Takeaways for Classifier Evaluation

- Never rely solely on the overall error rate: always inspect the **confusion matrix**.
- Report **sensitivity** and **specificity** separately — they reveal class-specific performance.
- Use the **ROC curve and AUC** to evaluate performance across all thresholds.
- Choose the decision threshold based on **domain knowledge** about the costs of each error type.

Part X

Tree-Based Methods

41 Introduction to Tree-Based Methods

41.1 Overview

Tree-Based Methods

Tree-based methods are a family of techniques for **regression** and **classification** that work by **stratifying** or **segmenting** the predictor space into a number of **simple regions**. The set of splitting rules used to produce the regions can be summarised in a **tree**, hence the name decision tree.

The core mechanics are:

- **Splitting rules:** a sequence of binary conditions on the predictors that recursively partition the input space.
- **Prediction for a new observation:** locate the region the observation falls into and return a **representative value** for that region (the mean of training responses for regression, the most frequent class for classification).
- **Tree representation:** the splitting rules can be drawn as a tree, making the model easy to visualise.

Decision trees are:

- **Simple and useful for interpretation.**
- Usually **not competitive** with other state-of-the-art methods in terms of prediction accuracy.
- Improvable by **combining many trees:** bagging, random forest, boosting.
- Subject to a fundamental **trade-off between prediction accuracy and interpretability:** a single tree is highly interpretable; ensembles of trees are far more accurate but lose interpretability.

41.2 Basic Terminology

A decision tree is composed of:

- **Terminal nodes** (or **leaves**): the regions $R_1, \dots, R_J$ into which the predictor space is partitioned. Each terminal node carries a representative value.
- **Internal nodes:** the split points; each is labelled with the **splitting condition** (e.g., $Years < 4.5$).
- **Root:** the top of the tree, where the recursion starts.

Example (Hitters dataset, predicting log Salary from Years and Hits). The tree has two internal nodes ($Years < 4.5$ and $Hits < 117.5$) and three terminal nodes corresponding to three regions:

$$R_1 = \{X \mid Years < 4.5\}, \quad R_2 = \{X \mid Years \geq 4.5, Hits < 117.5\}, \quad R_3 = \{X \mid Years \geq 4.5, Hits \geq 117.5\}$$

The predictions 5.11, 6.00, 6.74 are the mean log salary in each region. Interpretation: *Years* is the most important factor for salary; *Hits* plays a minor role for less experienced players but

becomes important for more experienced ones. The tree is an extremely **compact representation**, easier to interpret than a regression model.

42 Regression Trees

42.1 Tree-Building Process

Goal of Regression Trees

Divide the predictor space (with predictors $X_1, X_2, \dots, X_p$) into J **non-overlapping regions** $R_1, R_2, \dots, R_J$. For every observation that falls into a region R_j , make the **same prediction**: the **mean of the response values** of all training observations in R_j .

In principle the regions could be of any shape, but **boxes** (high-dimensional rectangles) are used to keep interpretation simple. The objective is to find boxes $\{R_1, \dots, R_J\}$ that minimise the **residual sum of squares**:

$$\text{RSS} = \sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2$$

where $\hat{y}_{R_j}$ is the mean response of the training observations within the j -th box.

42.2 Recursive Binary Splitting

It is computationally **infeasible** to consider every possible partition of the feature space into J boxes. Trees are therefore built with a **top-down, greedy** approach known as **recursive binary splitting**:

- **Top-down**: start from a single region (the whole space) and perform splits.
- **Binary**: each split produces exactly two regions.
- **Greedy**: each split optimises the current step only — no look-ahead.

Recursive Binary Splitting Step

Select the predictor X_j and the cutoff value s such that the RSS for the two regions

$$R_1(j, s) = \{X \mid X_j < s\}, \quad R_2(j, s) = \{X \mid X_j \geq s\}$$

is minimised:

$$\text{RSS}(j, s) = \sum_{i: x_i \in R_1(j, s)} (y_i - \hat{y}_{R_1(j, s)})^2 + \sum_{i: x_i \in R_2(j, s)} (y_i - \hat{y}_{R_2(j, s)})^2$$

The pair (j^*, s^*) that minimises $\text{RSS}(j, s)$ is selected. Finding (j^*, s^*) is fast when p is not too large.

After the first split, the procedure is **repeated** on one of the two resulting regions, then on the next, and so on. It continues until a **stopping criterion** is met (for example, no region contains more than O observations).

42.3 Predictions

To compute the response for a test observation:

1. Determine the region (terminal node) to which the test observation belongs by following the splits from the root.
2. Return the **mean of the training observations** for that region.

42.4 Pruning the Decision Tree

The recursive binary splitting procedure may **overfit** the training data, producing a tree with too many terminal nodes that captures noise rather than signal. There is a **trade-off between variance** (interpretability) **and bias**.

A natural idea is to design a stricter **stopping criterion** (e.g., split only if the RSS reduction is high). However, this can perform poorly because of the greedy nature of the procedure: a split that reduces RSS only by a small amount now may lead to subsequent splits that yield much larger reductions later.

Pruning Strategy

Build a very large tree T_0 first, then **prune it back** to obtain a smaller subtree $T \subset T_0$ with better generalisation.

42.5 Cost-Complexity (Weakest Link) Pruning

Cost-complexity pruning, also called **weakest link pruning**, is the standard pruning strategy. Consider a sequence of trees indexed by a non-negative tuning parameter $\alpha \geq 0$. For each value of α there is a subtree $T \subset T_0$ that minimises:

Cost-Complexity Criterion

$$\sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

where $|T|$ is the **number of terminal nodes** of the tree T .

Properties:

- As α increases, the procedure generates a **nested sequence of trees** $T_0 \supset T_1 \supset \dots \supset T_m$, where T_m contains only the root.
- Each tree in the sequence is a subtree of T_0 obtained by **collapsing internal nodes**.
- The best tree in the sequence is selected via **cross-validation**.

42.6 Regression Tree Procedure

Full Algorithm for Regression Trees

1. Use **recursive binary splitting** to build a large tree on the training data, with a stopping criterion on the number of observations on terminal nodes (usually small).
2. Apply **cost-complexity pruning** to obtain a sequence of subtrees as a function of α .
3. Use **K -fold cross-validation** to choose α . For each fold $k = 1, \dots, K$:
 - (a) Repeat steps 1 and 2 on all data except the k -th fold.
 - (b) Evaluate the mean squared prediction error on the k -th fold as a function of α .
4. **Average** the results across folds for each α and pick the α^* that minimises the average error.
5. Return the subtree from step 2 corresponding to α^* .

Example (Hitters dataset). Randomly divide the data into 132 training and 131 test observations. Build a large regression tree on the training data, prune via cost-complexity to obtain a series of subtrees indexed by α , then perform 6-fold cross-validation to determine α^* that minimises the cross-validated MSE. The training MSE keeps decreasing with tree size; the cross-validation and test MSE curves are U-shaped, indicating an optimal intermediate size.

43 Classification Trees

43.1 From Regression to Classification

A **classification tree** predicts a **qualitative** response. The structure is the same as a regression tree, but:

- The **prediction** for a region is the **most commonly occurring class** among the training observations in that region.
- Recursive binary splitting is still used, but **RSS cannot be the splitting criterion** (the response is qualitative, not quantitative).

A natural alternative is the **classification error rate**:

$$E = 1 - \max_k (\hat{p}_{mk})$$

where $\hat{p}_{mk}$ is the proportion of training observations in region m that belong to class k . However, classification error is not sensitive enough for tree-growing — small differences in class composition do not change E . Two more sensitive criteria are used instead.

43.2 Gini Index and Entropy

Gini Index

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$$

where K is the total number of classes.

- G is **small** if all $\hat{p}_{mk}$ are close to 0 or 1.
- G measures **node purity**: it is close to zero when a node contains predominantly observations from a single class.

Entropy

$$D = - \sum_{k=1}^K \hat{p}_{mk} \log \hat{p}_{mk}$$

The Gini index G and entropy D are usually **numerically very similar** and both behave smoothly. As a function of p (for two classes, p being the proportion of one class), entropy and Gini are concave curves peaking at $p = 0.5$, while the misclassification error is piecewise linear with a sharp peak. The smoothness of G and D is what makes them better for splitting decisions.

Example (Heart dataset). Binary outcome **HD** (Yes/No, presence of heart disease) for 303 patients with chest pain, with 13 predictors including *Age*, *Sex*, *Chol*. The pruned tree, selected by cross-validation, has 6 terminal nodes and uses splits on *Thal*, *Ca*, *MaxHR*, *ChestPain*, etc.

43.3 Linear Models vs. Tree-Based Approaches

The decision boundaries differ qualitatively:

- **Linear models:** produce **linear** (oblique) decision boundaries.
- **Tree-based methods:** produce **axis-aligned, piecewise-constant** decision boundaries.

When the true boundary is linear, linear models perform better; when the true boundary is highly non-linear and axis-aligned, trees do better.

43.4 Benefits and Drawbacks of Tree-Based Methods

Pros and Cons**Pros:**

- Trees are very **easy to explain** to non-technical audiences.
- Trees are very **easy to interpret** visually.
- Great tool to **support decisions**.
- Can directly **handle qualitative predictors** without dummy encoding.

Cons:

- **Not competitive** in prediction accuracy with state-of-the-art methods.

Predictive accuracy can be significantly increased by **aggregating many decision trees** via bagging, random forests, and boosting.

44 Bootstrapping and Bagging

44.1 The Bootstrap Concept

The **bootstrap** is a widely applicable statistical tool used to **estimate the uncertainty** of an estimator (or, more generally, of a statistical learning method).

Financial example. Two assets with expected returns X and Y . The expected return on an investment of fraction α in X and $1 - \alpha$ in Y is $\alpha X + (1 - \alpha)Y$, with variance $\text{Var}(\alpha X + (1 - \alpha)Y)$. The variance-minimising weight is:

$$\hat{\alpha}^* = \frac{\hat{\sigma}_Y^2 - \hat{\sigma}_{XY}}{\hat{\sigma}_X^2 + \hat{\sigma}_Y^2 - 2\hat{\sigma}_{XY}}$$

We want to know the **standard error** of $\hat{\alpha}$.

44.2 Estimating $\text{Var}(\hat{\alpha})$

Two approaches:

- **Synthetic resampling (population available):** generate 1000 independent synthetic data sets, estimate $\hat{\alpha}_r$ on each, and compute the standard deviation across the 1000 estimates. On the financial example: $SE(\hat{\alpha}) = 0.086$.
- **Bootstrap (only one data set available):** sample 1000 data sets **with replacement** from the single original data set, estimate $\hat{\alpha}$ on each, and compute the standard deviation. On the same example: $SE(\hat{\alpha}) = 0.087$ — **very similar** to the synthetic estimate.

Bootstrap Procedure

Given an original data set Z with n observations, repeat B times:

1. Draw a bootstrap sample Z^{*b} of size n **with replacement** from Z .
2. Compute the estimate $\hat{\alpha}^{*b}$ on Z^{*b} .

The empirical distribution of $\{\hat{\alpha}^{*1}, \dots, \hat{\alpha}^{*B}\}$ approximates the sampling distribution of $\hat{\alpha}$.

44.3 Bagging

Bootstrap Aggregation (Bagging)

Bagging is a general-purpose procedure for **reducing the variance** of a statistical learning method. It is frequently used with decision trees.

The intuition: given n independent observations $Z_1, \dots, Z_n$, each with variance σ^2 , the variance of their mean $\bar{Z}$ is σ^2/n . **Averaging reduces variance.** Applying this idea to learning:

1. Generate B different bootstrapped training sets.
2. Train a model on each bootstrap sample, obtaining $\hat{f}^{*1}, \dots, \hat{f}^{*B}$.
3. Average the predictions:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x)$$

44.4 Bagging for Decision Trees

Regression trees:

- Train B **deep trees** via recursive binary splitting, with stopping criteria tuned to grow deep trees and **no pruning**. The result is trees with **high variance but low bias**.
- For a new point x , compute the prediction from each tree and **average** the results.

Classification trees:

- Train B deep classification trees without pruning.
- For a new point x , compute the predicted class for each of the B trees and return the **most frequent class** (majority voting).

44.5 Out-Of-Bag (OOB) Error Estimation

OOB Error

On average, each bagged tree is built using about **two-thirds** of the training observations. The remaining **one-third** not used to fit a given tree are called the **out-of-bag (OOB) observations** for that tree.

To estimate the test error of a bagged model:

- For the i -th observation, predict its response using each of the trees for which it was OOB. This yields approximately $B/3$ predictions.
- **Average** (regression) or take a majority vote (classification) of those predictions.
- Compute the resulting error across all observations.

For large B , the OOB error is essentially the **leave-one-out cross-validation error** for bagging — obtained without performing CV explicitly.

44.6 Variable Importance

Bagging improves accuracy at the cost of **interpretability** (no single tree to inspect). A summary of predictor relevance can still be produced via a **variable importance measure**: for each predictor, record the total amount by which the RSS (regression) or the Gini index (classification) decreases due to splits on that predictor, averaged over all B trees. The values are usually expressed relative to the maximum.

Example (Heart dataset). Variable importance ranks *Thal*, *Ca*, *ChestPain*, and *Oldpeak* as the most relevant predictors.

45 Random Forest and Boosting

45.1 Random Forest

Random Forest

A random forest improves on bagged trees by **de-correlating** the trees. Like bagging, it builds many decision trees on bootstrapped training samples — but with a key change at each split.

Key difference from bagging: when building each tree (during recursive binary splitting), **each split is restricted to a random sample of m predictors** chosen out of the p available. A **fresh sample** of m predictors is drawn at every split. Typically:

$$m = \sqrt{p}$$

(e.g., $m = 4$ out of 13 for the Heart dataset). m should be smaller when there is a large number of **correlated predictors**.

Why de-correlate? Bagged trees built from a dataset with one or two very strong predictors will all use those predictors at the top split, producing trees that look similar and whose predictions are highly correlated. Averaging correlated quantities reduces variance much less than averaging uncorrelated ones. Restricting the split candidates forces some trees to use other predictors, making the trees less correlated and the average more effective.

Example (gene expression data). High-dimensional dataset with 4,718 gene expression measurements on 349 patients, each labelled with one of 15 classes (normal or one of 14 cancer types).

Selected the 500 genes with largest variance, split into training and test, and applied random forest with three values of m ($m = p$, $m = p/2$, $m = \sqrt{p}$). Smaller m values yield lower test classification error.

45.2 Boosting

Boosting

Boosting is another approach for improving decision trees, applicable to many statistical learning methods. Unlike bagging and random forest, boosting **does not use bootstrapping**: each tree is fit on a **modified version of the original data set**, and trees are grown **sequentially**, each using information from the previous trees.

45.3 Boosting Algorithm for Regression Trees

Boosting for Regression Trees

1. Set $\hat{f}(x) = 0$ and $r_i = y_i$ for all i in the training set.
2. For $b = 1, 2, \dots, B$:
 - (a) Fit a tree $\hat{f}^b$ with d splits ($d + 1$ terminal nodes) to the training data (X, r) .
 - (b) Update $\hat{f}$ by adding a shrunk version of the new tree:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x)$$

- (c) Update the residuals:

$$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i)$$

3. Output the boosted model:

$$\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}^b(x)$$

The model is fit to the **residuals** of the previous step, so each new tree focuses on what the current model misses.

45.4 Tuning Parameters for Boosting

1. **Number of trees B** : large B can result in **overfitting**, although the overfitting is usually slow. Use **cross-validation** to select B .
2. **Shrinkage parameter λ** : controls the **speed of learning**. Typical values are around 0.01 or 0.001. λ is related to B : **small λ requires large B** .
3. **Number of splits d** : number of splits in each tree, also called the **interaction depth**. Often $d = 1$ works well — each tree is a **stump** (single split), giving an **additive model**. More generally, d controls the **interaction order** of the boosted model: d splits can interact at most d variables.

Example (gene expression). Boosting with $\lambda = 0.01$ and depth 1 or 2 outperforms random forest ($m = \sqrt{p}$) on test classification error for large enough B .

45.5 Boosting for Classification: AdaBoost

Boosting for classification is similar but more involved. A classic example is **AdaBoost** (AdaBoost.M1, Freund and Schapire, 1997 — a.k.a. Discrete AdaBoost). Modern state-of-the-art

tree boosting methods (**XGBoost** and other **gradient boosting** systems) are based on the same principles, fitting a regularised additive model and using a second-order approximation in the optimisation step.

AdaBoost setup. Two-class problem $Y \in \{-1, +1\}$, predictors X , classifier $G(x)$. The error rate is:

$$\overline{\text{err}} = \frac{1}{N} \sum_{i=1}^N I(y_i \neq G(x_i))$$

A **weak classifier** is one whose error rate is only **slightly better than random guessing**. Boosting sequentially applies a weak classifier to repeatedly modified versions of the data, producing a series of weak classifiers $G_m(x)$, $m = 1, \dots, M$. The final prediction is a **weighted majority vote**:

$$G(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m G_m(x) \right)$$

The weights $\omega_1, \dots, \omega_N$ on the training observations (x_i, y_i) are updated to give **more importance to observations that are difficult to classify**.

AdaBoost.M1 Algorithm

1. **Initialise** observation weights $\omega_i = \frac{1}{N}$, $i = 1, \dots, N$.
2. For $m = 1, 2, \dots, M$:
 - (a) Fit a classifier $G_m(x)$ to the training data using weights ω_i .
 - (b) Compute the weighted error:

$$\text{err}_m = \frac{\sum_{i=1}^N \omega_i I(y_i \neq G_m(x_i))}{\sum_{i=1}^N \omega_i}$$

- (c) Compute $\alpha_m = \log \left(\frac{1 - \text{err}_m}{\text{err}_m} \right)$.
- (d) Update the weights:

$$\omega_i \leftarrow \omega_i \exp[-\alpha_m I(y_i \neq G_m(x_i))], \quad i = 1, \dots, N.$$

3. Output:

$$G(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m G_m(x) \right)$$

Example (simulated data, sum of 10 standard Gaussians). The weak classifier is a **stump** with 45% test error. AdaBoost drives the test error far below that of a 244-node tree as the number of boosting iterations grows, illustrating how aggregating weak learners can outperform a single complex model.

45.6 Summary

Tree-Based Methods: Key Takeaways

- Decision trees are **simple** and **interpretable** models for both regression and classification.
- Single trees are usually **not competitive** with other methods in terms of prediction accuracy.
- **Bagging**, **random forest**, and **boosting** significantly improve prediction accuracy by growing several trees and combining their predictions.
- **Random forest** and **boosting** are among the **state-of-the-art** approaches for supervised learning.
- There is always a fundamental **trade-off between interpretability and prediction accuracy**.

Part XI

Unsupervised Learning

46 Introduction to Unsupervised Learning

46.1 Refresher: Supervised, Unsupervised, Reinforcement Learning

The three main learning paradigms differ in what data is available:

- **Supervised learning:** $y = f(x)$ — given pairs (x, y) , learn $f(\cdot)$ [function approximation].
- **Unsupervised learning:** $f(x)$ — given only x , learn $f(\cdot)$ as a **compact representation** of x [clustering, dimension reduction].
- **Reinforcement learning:** $y = f(x)$, z — given (x, z) , learn $f(\cdot)$ that generates y , where x is the state and z is the reward signal.

46.2 Challenges of Unsupervised Learning

Why Unsupervised is Hard

- Usually **more challenging** than supervised learning: it is **more subjective**, with goals that are not clearly defined.
- **Difficult to assess results:** there is no universally accepted mechanism to perform cross-validation.
- Main issue: we do **not** have the labels $\Rightarrow$ we do **not have the ground truth**.

Despite the difficulties, unsupervised learning has many important applications:

- Find **similarities in genes** or in **breast cancer samples** (or both) to better understand the disease.
- Find **similarities in users** of online shopping or on-demand platforms for targeted advertisement.
- Find **similar behaviours in malware samples** to devise appropriate countermeasures.

47 Principal Component Analysis (Revised)

47.1 Overview

Principal Component Analysis (PCA) can:

- **Summarise** a set of highly correlated variables with a smaller number of **representative** variables.
- Identify **principal component directions** that **maximise the variability** of the data in feature space.
- Equivalently, find directions that are **as close as possible to the data**.

PCA is an **unsupervised approach** (no associated responses) and is very useful for **data visualisation**, particularly in high-dimensional spaces.

47.2 First Principal Component

Given a set of features $\{X_1, X_2, \dots, X_p\}$, the **first principal component** Z_1 is the **normalised linear combination** of the features that has the **largest variance**:

$$Z_1 = \phi_{11}X_1 + \phi_{21}X_2 + \dots + \phi_{p1}X_p, \quad \text{with} \quad \sum_{j=1}^p \phi_{j1}^2 = 1$$

The vector $\phi_1 = (\phi_{11}, \phi_{21}, \dots, \phi_{p1})^T$ is called the **loading vector**. The constraint $\sum_j \phi_{j1}^2 = 1$ is needed because, otherwise, one could trivially increase the variance by scaling up the loadings.

47.3 Computing the First Principal Component

Let $\mathbf{X}$ be the data matrix with n rows (observations) and p columns (features). PCA cares only about variance, so the **mean of each column is set to zero** (centring). The objective is to find a linear combination of the centred features that maximises the variance of the resulting **principal component scores**:

$$z_{i1} = \phi_{11}x_{i1} + \phi_{21}x_{i2} + \dots + \phi_{p1}x_{ip}, \quad \sum_{j=1}^p \phi_{j1}^2 = 1$$

Since each X_j has zero mean, so does Z_1 , hence $\text{Var}(Z_1) = \frac{1}{n} \sum_{i=1}^n z_{i1}^2$. The optimisation problem is therefore:

First PC Optimisation Problem

$$\max_{\phi_{11}, \phi_{21}, \dots, \phi_{p1}} \frac{1}{n} \sum_{i=1}^n (\phi_{11}x_{i1} + \phi_{21}x_{i2} + \dots + \phi_{p1}x_{ip})^2 \quad \text{subject to} \quad \sum_{j=1}^p \phi_{j1}^2 = 1$$

47.4 Further Principal Components

The **second principal component** Z_2 is the linear combination of $X_1, \dots, X_p$ that has **maximal variance** and is **uncorrelated** with Z_1 :

$$z_{i2} = \phi_{12}x_{i1} + \phi_{22}x_{i2} + \dots + \phi_{p2}x_{ip}, \quad \sum_{j=1}^p \phi_{j2}^2 = 1$$

Constraining Z_2 to be uncorrelated with Z_1 is equivalent to constraining the direction ϕ_2 to be **orthogonal** to ϕ_1 , and so on for higher components.

The problem is solved via the **Singular Value Decomposition (SVD)** of $\mathbf{X}$. The principal component directions $\phi_1, \phi_2, \dots, \phi_K$ are the ordered **eigenvectors** of $\mathbf{X}^T \mathbf{X}$, while the variances of the components are the corresponding **eigenvalues**. The maximum number of components is $K = \min\{n - 1, p\}$.

47.5 Example: USA Arrest Dataset

USA arrest data: for each of the 50 U.S. states, the number of arrests per 100,000 residents in three categories — *Assault*, *Murder*, *Rape* — plus a fourth feature *UrbanPop* (percentage of population in urban areas). So $n = 50$ score vectors and $p = 4$ loading vectors. PCA is performed after **data standardisation** (zero mean, unit standard deviation). The first two principal components separate states by overall crime rate (PC1) and by the urban/rape vs. murder/assault contrast (PC2).

47.6 PCA as the Closest Linear Surface

There are two equivalent interpretations:

- **Maximum variance:** the loading vectors are the directions in feature space along which the data **vary the most**.
- **Closest linear surface:** low-dimensional linear surfaces (lines, planes, ...) that are **closest to the observations** in average squared Euclidean distance.

Specifically, the **first principal component** is the line in p -dimensional space that is closest to the n observations, i.e., the single dimension that **best approximates** all observations. The first M principal components and their score vectors yield the best M -dimensional approximation of each observation:

$$x_{ij} \approx \sum_{m=1}^M z_{im} \phi_{jm}$$

47.7 Proportion of Variance Explained (PVE)

The strength of a component is summarised by the **Proportion of Variance Explained (PVE)** by that component. Assuming the data are centred, the **total variance** is:

$$\sum_{j=1}^p \text{Var}(X_j) = \sum_{j=1}^p \frac{1}{n} \sum_{i=1}^n x_{ij}^2$$

The **variance explained** by the m -th principal component is:

$$\text{Var}(Z_m) = \frac{1}{n} \sum_{i=1}^n z_{im}^2$$

When $M = \min\{n - 1, p\}$ all variance is captured:

$$\sum_{j=1}^p \text{Var}(X_j) = \sum_{m=1}^M \text{Var}(Z_m)$$

PVE Formula

$$\text{PVE}_m = \frac{\sum_{i=1}^n z_{im}^2}{\sum_{j=1}^p \sum_{i=1}^n x_{ij}^2}$$

47.8 How Many Components to Use?

There is **no simple answer**: PCA is unsupervised, so cross-validation is not directly applicable (no response variable, no test error to minimise). Cross-validation can be used only when PCA is followed by a supervised task (e.g., principal component regression). In purely unsupervised settings the most common heuristic is the **scree plot**: plot the PVE (or cumulative PVE) against the component index and **look for an elbow**, i.e., the component after which additional components contribute marginally to the explained variance.

47.9 Scaling the Variables

When variables are in **different units** it is **important to scale** them to have unit standard deviation before applying PCA. Otherwise, variables with larger numerical range will dominate the variance and bias the principal directions. Standardisation removes this artefact and makes the loading vectors comparable across features.

48 Clustering Methods: K-means Clustering

48.1 What is Clustering?

Clustering

A broad set of techniques for finding **subgroups** (also called **clusters**) in a data set. The goal is to **partition** the data into groups so that observations within each group are **similar to each other** and **different** from observations in other groups.

The key point is to specify what is meant by **similar** and **different**: similarity (or dissimilarity) is often a **domain-specific** concept based on knowledge of the data.

PCA vs. Clustering:

- **PCA** looks for a **low-dimensional representation** of the observations that explains a large fraction of the variance.
- **Clustering** looks for **homogeneous subgroups** among the observations.

Example (market segmentation). Given a large number of measurements (median household income, occupation, distance from urban areas, ...) for many people, identify subgroups of people who may be more receptive to a particular form of advertising or interested in a given product. Market segmentation $\Rightarrow$ clustering people in the data set.

48.2 Two Main Clustering Methods

- **K-means clustering**: partition observations into a **predefined number of clusters** K .
- **Hierarchical clustering**: build a **tree-like representation** of the observations, called a **dendrogram**, that allows visualisation of clusterings for **every possible number of clusters** $1, \dots, n$.

48.3 K-means: Setup

Let $C_1, \dots, C_K$ denote a partition of the indices of the observations. If observation i belongs to cluster k , then $i \in C_k$. The partition must satisfy:

- **Cover:** each observation belongs to at least one cluster:

$$\bigcup_{k=1}^K C_k = \{1, 2, \dots, n\}$$

- **Non-overlapping:** no observation belongs to more than one cluster:

$$\forall k, k' \mid k \neq k', \quad C_k \cap C_{k'} = \emptyset$$

48.4 Within-Cluster Variation and Objective

The **within-cluster variation** $WCV(C_k)$ measures how much the observations belonging to cluster C_k differ from each other. K-means seeks the partition that minimises the total within-cluster variation:

$$\arg \min_{C_1, \dots, C_K} \left\{ \sum_{k=1}^K WCV(C_k) \right\}$$

The form of WCV depends on the application. A popular choice is the **squared Euclidean distance**:

$$WCV(C_k) = \frac{1}{|C_k|} \sum_{i, i' \in C_k} \sum_{j=1}^p (x_{ij} - x_{i'j})^2$$

K-means Objective

$$\arg \min_{C_1, \dots, C_K} \left\{ \sum_{k=1}^K \frac{1}{|C_k|} \sum_{i, i' \in C_k} \sum_{j=1}^p (x_{ij} - x_{i'j})^2 \right\}$$

48.5 K-means Algorithm

K-means Algorithm

1. Randomly assign a number from $1, \dots, K$ to each observation. This is the **initial cluster assignment**.
2. **Iterate** until convergence (i.e., the cluster assignment does not change):
 - (a) For each cluster, compute the **cluster centroid**: the vector of p feature means for the observations in the k -th cluster.
 - (b) **Assign each observation to the closest centroid**, using Euclidean distance.

48.6 Properties of the K-means Algorithm

The algorithm is **guaranteed to decrease the objective at each step**. The key identity is:

$$\frac{1}{|C_k|} \sum_{i, i' \in C_k} \sum_{j=1}^p (x_{ij} - x_{i'j})^2 = 2 \sum_{i \in C_k} \sum_{j=1}^p (x_{ij} - \bar{x}_{kj})^2$$

where $\bar{x}_{kj} = \frac{1}{|C_k|} \sum_{i \in C_k} x_{ij}$ is the mean of feature j in cluster C_k . This shows that minimising pairwise squared distances within a cluster is equivalent to minimising squared distances to the centroid — which is exactly what step 2.b of the algorithm does.

Local Optima

The algorithm is **not guaranteed to find the global optimum**. Different random initialisations can lead to different solutions with different values of the objective. In practice, K-means is run multiple times with different initialisations and the solution with the lowest objective value is kept.

49 Hierarchical Clustering

49.1 Overview

Hierarchical Clustering

Hierarchical clustering builds and maintains a **hierarchy of clusters**. The most common variant is **bottom-up** or **agglomerative** clustering: a **dendrogram** is built from the leaves (the individual observations) upwards, combining clusters until a single cluster (the trunk) is reached.

49.2 Hierarchical Clustering Algorithm

Agglomerative Hierarchical Clustering

1. Start with **each point in its own cluster**.
2. Identify the **closest couple of clusters** and **merge them**.
3. Repeat from step 2 until all points are in a single cluster.

The result is a **dendrogram**: a tree where leaves are observations and the height of each merge represents the dissimilarity between the two merged clusters. **Cutting the dendrogram horizontally** at a chosen height yields a clustering: the higher the cut, the fewer clusters obtained.

Example. For 45 observations in two dimensions belonging to three classes (unknown to the algorithm), a dendrogram cut at height 9 produces 2 clusters; a cut at height 5 produces 3 clusters that closely match the true classes.

49.3 Types of Linkage

The notion of **distance between clusters** (rather than between individual points) is encoded by the **linkage**. The four main types are:

Linkage	Description
Complete	Maximal inter-cluster dissimilarity. Compute all pairwise dissimilarities between observations in cluster A and cluster B , record the largest .
Single	Minimal inter-cluster dissimilarity. Compute all pairwise dissimilarities between observations in A and B , record the smallest .
Average	Mean inter-cluster dissimilarity. Compute all pairwise dissimilarities between observations in A and B , record the average .
Centroid	Dissimilarity between the centroid of A (a mean vector of length p) and the centroid of B . Centroid linkage can produce undesirable inversions .

Linkage choice affects the shape of the dendrogram: **single linkage** tends to produce long, chained clusters; **complete** and **average** linkage produce more balanced, compact clusters and are usually preferred in practice.

49.4 Choice of Dissimilarity Measure

The dissimilarity measure has a strong impact on the result:

- **Euclidean distance:** the typical default.
- **Correlation-based distance:** two observations are considered similar if they are highly correlated, regardless of magnitude. Useful when the **shape** of the profile matters more than absolute values (e.g., gene expression patterns over time).

49.5 Practical Issues

Several practical decisions affect the outcome of clustering:

- **Scaling of variables:** should observations be **standardised** before clustering? Variables in larger units would otherwise dominate the dissimilarity.
- For hierarchical clustering: which **dissimilarity measure** and which **type of linkage** to use?
- How many clusters should be chosen?
- Which **features** should drive the clustering?

There is no universally correct answer — choices are guided by domain knowledge and validated by inspection of the results.

49.6 Example: Breast Cancer Microarray Study

Sorlie et al. (PNAS 2003) repeatedly observed breast tumour subtypes in independent gene-expression datasets. About **8000 genes** were measured for **88 breast cancer patients**; clustering used **average linkage** with a **correlation metric** on the **500 intrinsic genes** (those with the smallest within-patient/between-patient variation ratio). Each woman was measured before and after chemotherapy. Clustering revealed several tumour subtypes (Luminal A, Luminal B, ERBB2, Basal, Normal-Breast-like) that turned out to be associated with different survival probabilities ($p < 0.01$ in the Norway/Stanford data set), demonstrating that unsupervised clustering can reveal clinically relevant structure.

49.7 Conclusions

Unsupervised Learning: Key Takeaways

- Unsupervised learning is important for understanding the **variation** and **structure** of an unlabelled dataset.
- It can be a useful step **before** performing supervised learning.
- It is intrinsically **more difficult** than supervised learning: no ground truth (labels) and no single objective (no test prediction accuracy to minimise).
- It is a challenging and **active field of research**, with many recently developed tools: self-organising maps, spectral clustering, affinity propagation, and so on.